

MÁSTER DE FORMACIÓN PERMANENTE EN INTELIGENCIA
ARTIFICIAL

**Diseño e implementación de una
arquitectura modular basada en
Retrieval-Augmented Generation (RAG)
para la creación y despliegue configurable de
chatbots de atención al cliente**

Autor:

ADE CABO CUARTERO

Tutor:

VICENTE CASTILLO FAULI

CURSO ACADÉMICO 2025–2026

Resumen

La adopción de Modelos de Lenguaje de Gran Escala (LLMs) en chatbots de atención al cliente está limitada por su incapacidad para acceder a información privada y actualizada, lo que provoca alucinaciones y respuestas poco fiables. El paradigma de Generación Aumentada por Recuperación (RAG) mitiga este problema, pero sus implementaciones actuales se desarrollan a medida, requieren conocimientos técnicos avanzados y resultan difícilmente reutilizables; las plataformas comerciales que simplifican el proceso lo hacen a costa de ser cerradas y dependientes del proveedor. Este trabajo presenta el diseño e implementación de una plataforma de código abierto, autoalojable y genuinamente modular que permite crear, configurar y desplegar chatbots basados en RAG a través de un panel de administración web, sin necesidad de programar. La plataforma integra fuentes de conocimiento heterogéneas —archivos locales, almacenamiento en la nube y bases de datos SQL— mediante un módulo de enrutamiento que decide entre búsqueda semántica sobre documentos vectorizados, generación de consultas text-to-SQL, o una combinación de ambas. La arquitectura, basada en el patrón de puertos y adaptadores, formaliza contratos públicos para cada componente del pipeline —modelo de lenguaje, sistema de embeddings, base de datos vectorial, fuentes de datos—, de modo que cualquier proveedor pueda sustituirse por una implementación propia sin modificar el núcleo del sistema si así se desea. La estrategia de fragmentación, junto con otros parámetros del pipeline, queda expuesta al administrador como configuración del chatbot. El sistema se evalúa cuantitativamente mediante métricas estandarizadas de recuperación y generación y se demuestra desplegando chatbots de atención al cliente en escenarios diferenciados que cubren las distintas modalidades de recuperación soportadas.

Palabras clave: RAG, chatbot, arquitectura modular, LLM, puertos y adaptadores, código abierto, atención al cliente

Abstract

The adoption of Large Language Models (LLMs) in customer service chatbots is limited by their inability to access private, up-to-date information, leading to hallucinations and unreliable responses. The Retrieval-Augmented Generation (RAG) paradigm mitigates this issue, but current implementations are built on a case-by-case basis, require advanced technical expertise, and are difficult to reuse; commercial platforms that simplify the process do so at the cost of being closed and vendor-dependent. This work presents the design and implementation of an open-source, self-hostable, and genuinely modular platform that enables users to create, configure, and deploy RAG-based chatbots through a web-based administration panel, with no programming required. The platform integrates heterogeneous knowledge sources —local files, cloud storage, and SQL databases— through a query routing module that decides between semantic search over vectorized documents, text-to-SQL query generation, or a combination of both. The architecture, based on the ports and adapters pattern, formalizes public contracts for each component of the pipeline —language model, embedding system, vector database, data sources—, so that any provider can be replaced by a custom implementation without modifying the core system. The chunking strategy and other pipeline parameters are exposed to the administrator as chatbot configuration. The system is evaluated quantitatively using standardized retrieval and generation metrics, and is demonstrated by deploying customer service chatbots in differentiated scenarios covering the distinct retrieval modalities supported.

Key words: RAG, chatbot, modular architecture, LLM, ports and adapters, open source, customer service

Índice general

Resumen	1
Abstract	2
1. Introducción	8
2. Objetivos	10
2.1. Objetivo general	10
2.2. Objetivos específicos	10
2.3. Alcance del trabajo	11
3. Marco teórico	12
3.1. Modelos de Lenguaje de Gran Escala (LLMs)	12
3.1.1. Definición y fundamentos	12
3.1.2. La arquitectura Transformer	13
3.1.3. Capacidades y tareas cognitivas	14
3.1.4. Componentes adicionales en aplicaciones comerciales	15
3.1.5. Limitaciones fundamentales	16
3.2. Retrieval-Augmented Generation (RAG)	18
3.2.1. Definición y fundamentos	18
3.2.2. Fase de indexación	19
3.2.3. Fase de recuperación y generación	21
3.2.4. Variantes y evolución	24
3.2.5. Limitaciones y desafíos	24
3.3. Embeddings y bases de datos vectoriales	26
3.3.1. Definición y fundamentos	26
3.3.2. Embeddings a nivel de oración y documento	26
3.3.3. Métricas de similitud	27
3.3.4. Bases de datos vectoriales	27
3.3.5. Algoritmos de indexación y búsqueda aproximada	28
3.4. Arquitectura de software modular	29
3.4.1. Definición y fundamentos	29
3.4.2. El patrón de puertos y adaptadores	29
3.4.3. Patrones complementarios	32

3.4.4.	Conexión con la implementación	32
3.5.	Estado del arte y trabajos relacionados	32
3.5.1.	Frameworks de orquestación RAG	32
3.5.2.	Plataformas comerciales de chatbots	33
3.5.3.	Plataformas open-source de bajo código	34
3.5.4.	Avances recientes en el paradigma RAG	34
3.5.5.	Avances recientes en modelos de embeddings	35
3.5.6.	Posicionamiento del presente trabajo	35
4.	Metodología	38
4.1.	Enfoque y tipo de investigación	38
4.2.	Fases del desarrollo	40
4.2.1.	Fase 1: Definición del problema y objetivos	40
4.2.2.	Fase 2: Marco teórico y metodología	40
4.2.3.	Fase 3: Diseño detallado, implementación y validación	40
4.2.4.	Fase 4: Redacción final y depósito	41
4.3.	Herramientas y tecnologías	41
4.3.1.	Backend: Python y FastAPI	42
4.3.2.	Frontend: Next.js	42
4.3.3.	Base de datos vectorial: Qdrant	42
4.3.4.	Base de datos relacional: PostgreSQL	42
4.3.5.	Modelos de lenguaje: Ollama y endpoints compatibles	43
4.3.6.	Infraestructura: Docker y Docker Compose	43
4.3.7.	Control de versiones y calidad del código	43
4.4.	Estrategia de evaluación	45
4.4.1.	Métricas de recuperación	45
4.4.2.	Métricas de generación	45
4.4.3.	Protocolo de evaluación	46
4.4.4.	Baselines y comparativas entre configuraciones	46
4.4.5.	Métricas operacionales	47
4.4.6.	Evaluación adversarial del módulo <i>text-to-SQL</i>	47
4.4.7.	Limitaciones y amenazas a la validez	48
4.4.8.	Reproducibilidad	49
4.5.	Datos y escenarios de validación	49
4.5.1.	Escenario 1: Atención al cliente sobre documentación de producto	49
4.5.2.	Escenario 2: Atención al cliente sobre catálogo y pedidos	50
4.5.3.	Escenario 3: Atención al cliente multi-fuente con enrutamiento	50
4.5.4.	Generación del dataset de evaluación	51
4.6.	Gestión de riesgos	51
5.	Desarrollo y Resultados	54
6.	Objetivos de Desarrollo Sostenible	55

7. Conclusiones	56
Referencias	57
Acrónimos	64
Anexos	66

Índice de figuras

3.1. Arquitectura del modelo Transformer. Fuente: Vaswani et al. (2017).	14
3.2. Visión general del pipeline RAG. La fase de indexación (superior) transforma los documentos en embeddings almacenados en una base de datos vectorial. La fase de recuperación (inferior) convierte la consulta del usuario en un embedding, recupera los fragmentos más relevantes y los proporciona como contexto al LLM para generar la respuesta.	19
3.3. Enrutamiento de consultas en un sistema RAG multi-fuente. El enrutador analiza la consulta y la dirige hacia la estrategia de recuperación apropiada: búsqueda semántica para documentos no estructurados o generación de SQL para datos tabulares. Los resultados de ambas vías se proporcionan como contexto al LLM.	23
3.4. Aplicación del patrón de puertos y adaptadores a la plataforma. Elaboración propia basada en Cockburn (2005).	31

Índice de cuadros

3.1. Comparativa entre soluciones existentes y la plataforma propuesta.	36
4.1. Resumen del stack tecnológico de la plataforma.	44

Capítulo 1

Introducción

Durante los últimos años, los avances en los Modelos de Lenguaje de Gran Escala (LLMs) han transformado la forma en que las personas interactúan con los sistemas de información (Brown et al., 2020; Bommasani et al., 2021). A raíz de estos avances, organizaciones de todos los tamaños han comenzado a incorporar estos modelos en aplicaciones de chatbot para automatizar tareas como la atención al cliente, la resolución de consultas internas o el acceso a información y documentación corporativa. Sin embargo, estos modelos de lenguaje poseen un conocimiento estático, general y muchas veces desactualizado que adquieren durante su entrenamiento. Este es el principal obstáculo que dificulta su aplicación en entornos reales: al no tener acceso a la información privada y específica de cada organización, los modelos son demasiado genéricos para aplicaciones concretas y tienden a inventar información que no conocen («alucinaciones»), lo que erosiona la confianza del usuario final y limita su utilidad práctica.

Es aquí donde entra en juego el paradigma de la Generación Aumentada por Recuperación (RAG) (Lewis et al., 2020), que permite al sistema consultar fuentes de conocimiento externas —documentos, bases de conocimiento o bases de datos— para obtener contexto e información específica relevante antes de generar una respuesta. En los últimos años, la mayoría de empresas que han tratado de incorporar la inteligencia artificial han desarrollado estos sistemas para sus aplicaciones de reservas, ventas, gestores documentales, entre otros. Sin embargo, cada una de estas implementaciones se desarrolla a medida: la empresa necesita conectar sus fuentes de información, configurar el pipeline de procesamiento de documentos y desplegar la solución desde cero. Este enfoque artesanal requiere conocimientos técnicos avanzados, consume recursos desproporcionados respecto al valor que se obtiene y genera soluciones difícilmente reutilizables. Además, la dependencia de proveedores concretos de modelos de lenguaje o bases de datos vectoriales dificulta la migración y la evolución del sistema a largo plazo.

Este trabajo propone el diseño e implementación de una plataforma de código abierto y autoalojable que se encargue de todo este trabajo intermedio, de modo que el administrador pueda crear, configurar y desplegar chatbots íntegramente a través de un panel de administración web sin necesidad de programar. La plataforma gestiona internamente la ingestión y vectorización de documentos, el enrutamiento de consultas hacia la fuente

de datos apropiada y la generación de respuestas fundamentadas, mientras que el administrador tan solo debe enlazar sus fuentes de conocimiento —bases de datos SQL, almacenamiento en la nube o documentos (PDF, DOCX, TXT, entre otros)— y definir el contexto del chatbot (por ejemplo, «Eres el asistente virtual de la tienda online de la empresa»).

La arquitectura, basada en el patrón de puertos y adaptadores, organiza el sistema en módulos genuinamente independientes: cada componente del pipeline —modelo de lenguaje, sistema de embeddings, base de datos vectorial, fuentes de datos— opera con interfaces de entrada y salida definidas, de modo que el administrador puede utilizar los módulos predeterminados, sustituir cualquiera de ellos por una implementación propia ejecutada en su propia infraestructura, o incluso conectar un modelo de lenguaje local. Esta modularidad real es lo que diferencia la plataforma de las soluciones comerciales existentes, que ofrecen configurabilidad dentro de un sistema cerrado pero no permiten desacoplar ni reemplazar sus componentes internos.

El documento se estructura como sigue: el Capítulo 2 presenta los objetivos del trabajo; el Capítulo 3 desarrolla el marco teórico que fundamenta las decisiones de diseño; el Capítulo 4 describe la metodología seguida; el Capítulo 5 detalla el desarrollo y los resultados obtenidos; el Capítulo 6 analiza la relación con los Objetivos de Desarrollo Sostenible; y el Capítulo 7 recoge las conclusiones y líneas de trabajo futuro.

Capítulo 2

Objetivos

2.1. Objetivo general

Diseñar e implementar una plataforma de código abierto, autoalojable y modular, basada en el paradigma de Generación Aumentada por Recuperación (RAG), que permita crear, configurar y desplegar múltiples chatbots de forma autónoma, integrando diversos modelos de lenguaje —incluyendo modelos locales— y fuentes de conocimiento heterogéneas —documentos, almacenamiento en la nube y bases de datos SQL— bajo una arquitectura en la que cada módulo del pipeline sea independiente, sustituible y extensible por el administrador o por desarrolladores externos.

2.2. Objetivos específicos

1. **OE-1. Arquitectura modular con componentes desacoplables.** Diseñar una arquitectura basada en el patrón de puertos y adaptadores que organice el pipeline RAG en módulos independientes con interfaces de entrada y salida definidas, de modo que cualquier módulo —modelo de lenguaje, sistema de embeddings, base de datos vectorial— pueda ser sustituido por una implementación alternativa (incluyendo implementaciones propias del administrador o de un desarrollador externo, ejecutadas en su infraestructura) sin modificar el núcleo del sistema ni los demás módulos.
2. **OE-2. Pipeline de ingestión multi-fuente.** Implementar un pipeline capaz de procesar y vectorizar información procedente de tres tipos de fuentes: archivos locales (PDF, DOCX, TXT, CSV, Markdown), directorios en almacenamiento en la nube y bases de datos SQL, normalizando el contenido para su almacenamiento en una base de datos vectorial.
3. **OE-3. Motor RAG con enrutamiento de consultas.** Desarrollar el módulo de orquestación RAG que, ante una consulta del usuario final, determine la estrategia de recuperación apropiada —búsqueda semántica sobre documentos vectorizados,

generación y ejecución de consultas SQL, o una combinación de ambas— y construya el contexto necesario para que el modelo de lenguaje genere una respuesta fundamentada. El módulo incorpora además mecanismos de seguridad por capas que garantizan que las consultas SQL generadas dinámicamente no comprometan la integridad de las bases de datos consultadas.

4. **OE-4. Panel de administración web.** Desarrollar una interfaz web que permita crear, configurar y gestionar chatbots íntegramente a través del panel —definir instrucciones del sistema, seleccionar el modelo de lenguaje, vincular fuentes de conocimiento y ajustar los parámetros del pipeline RAG (estrategia de fragmentación, número de documentos recuperados, umbral de similitud)— sin necesidad de modificar el código fuente de la plataforma.
5. **OE-5. Evaluación del rendimiento del sistema RAG.** Evaluar la calidad del sistema mediante métricas estandarizadas del framework RAGAS: métricas de recuperación (*Context Precision*, *Context Recall*) y de generación (*Faithfulness*, *Answer Relevancy*), analizando el impacto de las distintas configuraciones —estrategias de fragmentación, modelos de embeddings y modelos de lenguaje— sobre la capacidad del sistema para generar respuestas correctas y mitigar alucinaciones.
6. **OE-6. Validación en escenarios de uso diferenciados.** Validar la versatilidad de la plataforma desplegando chatbots de atención al cliente en tres escenarios distintos que cubran las modalidades de recuperación soportadas —por ejemplo, un asistente sobre documentación de producto mediante búsqueda semántica y un asistente sobre catálogo o pedidos mediante *text-to-SQL*— para verificar que la arquitectura se adapta a dominios y tipos de fuentes de datos diferentes sin requerir modificaciones en el código.

2.3. Alcance del trabajo

El presente trabajo se centra en el diseño, implementación y evaluación de un producto mínimo viable (MVP) de la plataforma. Quedan fuera del alcance funcionalidades como el dashboard de analytics avanzado, el soporte multi-idioma nativo, la integración con canales de mensajería externos (WhatsApp, Telegram, Slack) y el fine-tuning de modelos desde la plataforma. No obstante, la arquitectura modular y de código abierto se diseña precisamente para que estas extensiones sean incorporables en futuras iteraciones —ya sea por el autor o por terceros— sin reestructurar el sistema.

Capítulo 3

Marco teórico

Este capítulo presenta los fundamentos teóricos que sustentan las decisiones de diseño de la plataforma. Se abordan en primer lugar los Modelos de Lenguaje de Gran Escala y sus limitaciones en entornos empresariales (Sección 3.1); a continuación, el paradigma de Generación Aumentada por Recuperación como respuesta a dichas limitaciones (Sección 3.2); los embeddings y las bases de datos vectoriales como infraestructura técnica subyacente (Sección 3.3); la arquitectura de software modular y el patrón de puertos y adaptadores como principio organizativo del sistema (Sección 3.4); y, finalmente, el estado del arte y los trabajos relacionados que contextualizan la contribución del presente trabajo (Sección 3.5).

La revisión bibliográfica se ha realizado consultando las bases de datos académicas Google Scholar, ACL Anthology, IEEE Xplore y arXiv, complementadas con la documentación oficial de las plataformas y herramientas analizadas. Los criterios de inclusión han sido la relevancia directa sobre los conceptos centrales del trabajo (LLMs, RAG, arquitectura hexagonal y bases de datos vectoriales), la actualidad de las fuentes —priorizando publicaciones de los últimos cinco años para los aspectos técnicos en evolución activa— y la combinación equilibrada entre referencias seminales (Vaswani 2017, Lewis 2020, Cockburn 2005) y *surveys* recientes que sintetizan el estado del arte. Para los modelos y plataformas comerciales sin publicación académica formal, se ha recurrido a la documentación oficial y a los repositorios de código abierto como fuentes primarias.

3.1. Modelos de Lenguaje de Gran Escala (LLMs)

3.1.1. Definición y fundamentos

Los Modelos de Lenguaje de Gran Escala (*Large Language Models*, LLMs) son modelos de aprendizaje profundo que han aprendido a modelar la complejidad del lenguaje natural mediante el entrenamiento con enormes volúmenes de datos textuales y una capacidad de cómputo sin precedentes. Estos modelos poseen desde cientos de millones hasta varios billones de parámetros, lo que les permite capturar patrones lingüísticos, relaciones semánticas y conocimiento factual de forma implícita en sus pesos internos. [Petroni](#)

et al. (2019) introdujeron la idea de que los LLMs funcionan como bases de conocimiento implícitas, almacenando lo aprendido en sus parámetros como memoria interna y generando respuestas mediante la recuperación de esa información, una observación que ha sido posteriormente confirmada y ampliada por trabajos sobre RAG (Wu et al., 2024).

De forma simplificada, el funcionamiento de un LLM se fundamenta en un proceso de predicción secuencial: el modelo recibe una entrada textual que se descompone en unidades llamadas *tokens* (palabras, subpalabras o caracteres) y, a partir de esa secuencia, genera el siguiente token mediante distribuciones probabilísticas. Cada nuevo token generado depende de todos los anteriores, lo que permite al modelo producir texto coherente y contextualmente relevante. Esta capacidad de generación secuencial es la base de todas las aplicaciones actuales de los LLMs, desde la respuesta a preguntas hasta la generación de código o la traducción automática.

3.1.2. La arquitectura Transformer

El avance fundamental que hizo posibles los LLMs modernos fue la introducción de la arquitectura *Transformer* (Vaswani et al., 2017). Frente a las arquitecturas recurrentes anteriores (RNN, LSTM), que procesaban las secuencias de texto palabra a palabra de forma secuencial, el Transformer introdujo el mecanismo de **autoatención** (*self-attention*), que permite al modelo evaluar simultáneamente la relevancia de cada token respecto a todos los demás tokens de la secuencia.

Este mecanismo es especialmente relevante para tareas de comprensión del lenguaje, ya que permite capturar dependencias a larga distancia —por ejemplo, relacionar un pronombre con el sustantivo al que hace referencia varios párrafos antes— sin la degradación de señal que sufren las arquitecturas recurrentes. Además, la paralelización inherente del Transformer permite un entrenamiento significativamente más eficiente, lo que posibilitó el escalado de los modelos a los tamaños actuales.

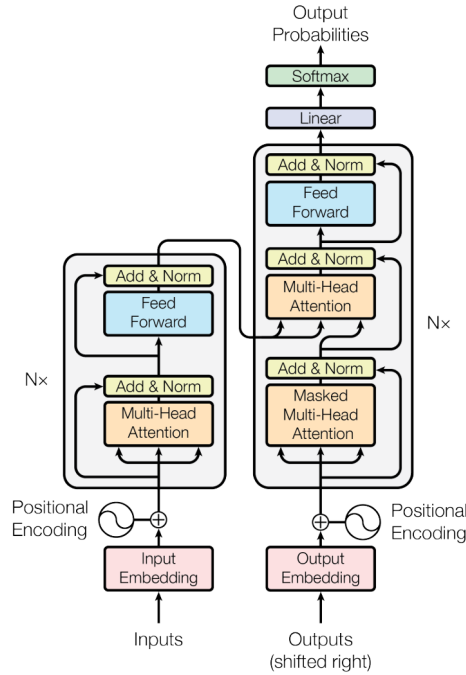


Figura 3.1: Arquitectura del modelo Transformer. Fuente: [Vaswani et al. \(2017\)](#).

Como muestra la Figura 3.1, la arquitectura original consta de un codificador (izquierda) que procesa la secuencia de entrada y un decodificador (derecha) que genera la salida. Ambos componentes se construyen apilando bloques que combinan capas de autoatención multi-cabezal (*Multi-Head Attention*) con redes *feed-forward*, conectadas mediante normalización y conexiones residuales. A partir de esta arquitectura base se han desarrollado dos familias principales de modelos relevantes para los sistemas conversacionales:

- **Modelos autoregresivos** (*decoder-only*): predicen el siguiente token dada la secuencia anterior. Es la arquitectura dominante en los LLMs generativos actuales, como los modelos GPT ([Brown et al., 2020](#)), Claude de Anthropic o Llama ([Touvron et al., 2023](#)).
- **Modelos de codificación** (*encoder-only*): procesan la secuencia completa de entrada para producir representaciones contextuales de cada token. BERT ([Devlin et al., 2019](#)) y sus variantes (RoBERTa, DistilBERT, ELECTRA) siguen este paradigma, y son especialmente utilizados en tareas de clasificación y generación de embeddings ([Reimers and Gurevych, 2019](#)).

3.1.3. Capacidades y tareas cognitivas

Los LLMs actuales han demostrado capacidad para automatizar diversas tareas cognitivas que tradicionalmente requerían intervención humana. [Wulf and Meierhofer \(2024\)](#) identifican cinco categorías de tareas cognitivas que estos modelos pueden abordar en contextos de atención al cliente:

1. **Traducción y corrección:** los LLMs pueden traducir texto entre idiomas o registros lingüísticos gracias a su comprensión de los patrones estructurales de múltiples lenguas.
2. **Resumen:** la capacidad de comprender las relaciones entre las distintas partes de un texto permite generar resúmenes que reflejan el significado global del documento original.
3. **Generación de contenido:** desde correos electrónicos hasta documentación técnica, los LLMs pueden producir texto original adaptado a diferentes formatos y tonos.
4. **Respuesta a preguntas:** el modelo utiliza el conocimiento factual adquirido durante el entrenamiento o la información contextual proporcionada en el prompt para responder consultas.
5. **Razonamiento:** a diferencia de la respuesta directa, el razonamiento complejo requiere la comprensión y aplicación de evidencia y lógica para alcanzar conclusiones a través de múltiples pasos.

Esta taxonomía es particularmente relevante para el presente trabajo, ya que la plataforma propuesta se centra en las tareas de respuesta a preguntas y razonamiento —precisamente las que presentan mayores dificultades cuando el modelo solo dispone de su conocimiento interno. Los resultados empíricos de [Wulf and Meierhofer \(2024\)](#) confirman que, mientras las tareas de nivel inferior (traducción, resumen, generación) pueden automatizarse de forma efectiva con los LLMs actuales, las tareas de nivel superior (razonamiento sobre datos específicos) requieren enfoques complementarios como la Generación Aumentada por Recuperación.

3.1.4. Componentes adicionales en aplicaciones comerciales

Antes de su uso en aplicaciones comerciales, los LLMs base —entrenados únicamente para predecir el siguiente token sobre grandes corpus de texto— atraviesan una fase de **ajuste por instrucciones** (*instruction tuning*) que los transforma en modelos capaces de seguir indicaciones del usuario, combinando aprendizaje supervisado sobre pares pregunta-respuesta de alta calidad (*Supervised Fine-Tuning*) y aprendizaje por refuerzo con retroalimentación humana (*Reinforcement Learning from Human Feedback*, RLHF) ([Ouyang et al., 2022](#)). La distinción entre el modelo base y el modelo ajustado es relevante: las aplicaciones empresariales utilizan exclusivamente versiones ajustadas (los *chat models* ofrecidos por OpenAI, Anthropic, Google o las variantes *instruct* de los modelos abiertos), porque solo estas son capaces de respetar instrucciones contextuales, mantener un tono coherente y rechazar peticiones inadecuadas.

Más allá del modelo ajustado, los sistemas comerciales emplean dos tipos de mecanismos para condicionar el comportamiento del modelo: **técnicas de prompting** (formas

concretas de redactar la entrada para inducir un tipo determinado de respuesta) y **capacidades estructurales del modelo** (extensiones genuinamente externas al texto del prompt que amplían lo que el modelo puede hacer).

Entre las técnicas de prompting más habituales destacan:

- **Instrucciones contextuales** (*system prompts*): directivas inyectadas al inicio de cada conversación que definen el rol, el tono y las restricciones del modelo antes de interactuar con el usuario final. Por ejemplo, «Eres un asistente de soporte técnico para la empresa X. Responde únicamente sobre los productos del catálogo». Aunque algunos sistemas tratan estas directivas como un canal estructural separado del mensaje del usuario, mecánicamente son texto que se concatena al *prompt* antes de la generación.
- **Cadena de razonamiento** (*chain-of-thought*) (Wei et al., 2022): induce al modelo a descomponer problemas complejos en pasos intermedios antes de emitir una respuesta final, mejorando la precisión en tareas que requieren lógica multi-paso. Esta descomposición se logra típicamente con instrucciones en el propio *prompt* («razona paso a paso») o proporcionando ejemplos resueltos previos (*few-shot prompting*).

La capacidad estructural más relevante para los sistemas conversacionales actuales es el **uso de herramientas** (*tool use* o *function calling*): la habilidad del modelo de invocar funciones externas —consultar una API, ejecutar código, realizar búsquedas— para complementar su conocimiento paramétrico con información obtenida en tiempo real. A diferencia de las técnicas de prompting, esta capacidad no es solo una forma de redactar la entrada, sino un protocolo estructural mediante el cual el modelo emite llamadas formales a funciones registradas y consume sus resultados antes de generar la respuesta final.

Estos mecanismos son relevantes para la plataforma propuesta en este trabajo, donde las instrucciones contextuales configuradas por el administrador definen el comportamiento de cada chatbot, y el uso de herramientas se materializa en la consulta a las fuentes de conocimiento a través del pipeline RAG.

3.1.5. Limitaciones fundamentales

A pesar de sus capacidades, los LLMs presentan limitaciones estructurales que condicionan su aplicación directa en entornos empresariales:

- **Conocimiento estático y desactualizado:** el conocimiento de un LLM queda fijado en el momento de su entrenamiento. Cualquier evento, actualización de producto o cambio normativo posterior a la fecha de corte del entrenamiento resulta inaccesible para el modelo. Reentrenar o ajustar un LLM para actualizar este conocimiento es un proceso costoso en tiempo y recursos computacionales (Wu et al., 2024).

- **Alucinaciones:** cuando un LLM no dispone de información suficiente para responder una consulta, tiende a generar respuestas que son lingüísticamente coherentes y fluidas pero factualmente incorrectas (Huang et al., 2023). Este fenómeno, conocido como *alucinación*, es especialmente problemático en aplicaciones donde la precisión de la información es crítica, como la atención al cliente o la consulta de documentación técnica. Como señalan Lewis et al. (2020), fundamentar las respuestas en conocimiento factual real reduce significativamente estas alucinaciones.
- **Ausencia de conocimiento privado o especializado:** los LLMs se entrenan con datos públicos de Internet y no tienen acceso a información corporativa, bases de datos internas ni documentación privada de una organización. Esto los hace demasiado genéricos para aplicaciones donde se requiere conocimiento específico del dominio.
- **Falta de trazabilidad:** las respuestas de un LLM generadas exclusivamente a partir de su conocimiento paramétrico no incluyen la procedencia de la información utilizada, lo que dificulta la verificación y genera desconfianza en entornos donde la fiabilidad es un requisito.
- **Coste y latencia operacionales:** la inferencia de un LLM grande tiene un coste no trivial por consulta, ya sea en consumo de *tokens* en una API comercial o en uso de hardware en una ejecución local. La latencia, además, escala con la longitud de la entrada y de la salida, y puede convertirse en un factor crítico en aplicaciones interactivas. Estos costes condicionan decisiones de diseño como la selección del tamaño del modelo, la cantidad de contexto que se incluye en cada llamada o el grado de *caching* aplicable.
- **Ventana de contexto finita:** aunque las ventanas de contexto se han extendido sustancialmente (de unos pocos miles de *tokens* en los primeros modelos GPT a cientos de miles e incluso millones en los modelos más recientes), siguen siendo finitas. Esta limitación es especialmente relevante en aplicaciones que requieren procesar grandes volúmenes de información: cualquier corpus que exceda la ventana del modelo necesita un mecanismo de selección que decida qué partes son relevantes para cada consulta, lo que motiva precisamente el paradigma RAG.

Estas limitaciones constituyen la motivación principal del paradigma de Generación Aumentada por Recuperación (RAG), que se analiza en la siguiente sección. En lugar de depender exclusivamente del conocimiento almacenado en los parámetros del modelo, RAG proporciona al LLM acceso a fuentes de información externas y actualizadas en el momento de la inferencia, abordando de forma simultánea los problemas de conocimiento estático, alucinaciones y falta de datos específicos del dominio (Lewis et al., 2020).

3.2. Retrieval-Augmented Generation (RAG)

3.2.1. Definición y fundamentos

La Generación Aumentada por Recuperación (*Retrieval-Augmented Generation*, RAG) es un paradigma que combina las capacidades generativas de los LLMs con sistemas de recuperación de información externa, permitiendo que el modelo fundamente sus respuestas en datos concretos y actualizados en lugar de depender exclusivamente de su **conocimiento paramétrico** —el conocimiento implícito que el modelo absorbe durante el entrenamiento y queda codificado en los pesos de su red neuronal—. El concepto fue formalizado por [Lewis et al. \(2020\)](#), quienes propusieron un modelo híbrido que integra un componente de recuperación —encargado de localizar documentos relevantes en una base de conocimiento— con un modelo generativo que utiliza dichos documentos como contexto para producir la respuesta final.

La motivación fundamental de RAG surge de las limitaciones de los LLMs descritas en la sección anterior. Como señalan [Gao et al. \(2024b\)](#), RAG aborda simultáneamente tres problemas: el conocimiento estático del modelo se complementa con fuentes actualizables; las alucinaciones se reducen al fundamentar las respuestas en evidencia recuperada; y el acceso a datos privados o especializados se resuelve sin necesidad de reentrenar el modelo. Además, a diferencia del *fine-tuning*, que requiere un proceso costoso de reentrenamiento cada vez que cambian los datos, RAG permite actualizar el conocimiento del sistema simplemente modificando los documentos almacenados en la base de conocimiento.

El funcionamiento general de un sistema RAG se divide en dos grandes fases que operan de forma complementaria: una **fase de indexación**, que se ejecuta de forma anticipada y prepara las fuentes de conocimiento para ser consultadas eficientemente; y una **fase de recuperación y generación**, que se activa en tiempo real ante cada consulta del usuario. La Figura 3.2 muestra una visión general de ambas fases. Las siguientes subsecciones detallan cada una de ellas.

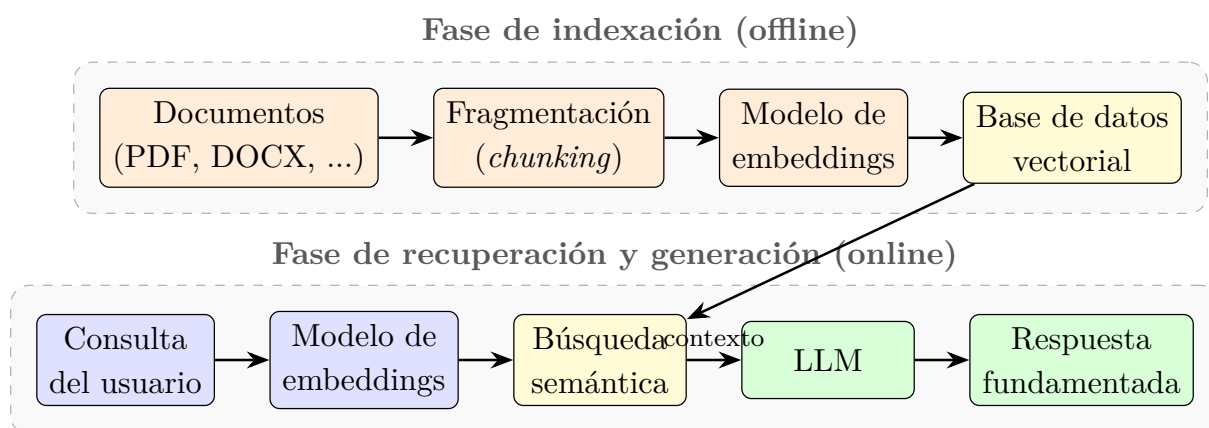


Figura 3.2: Visión general del pipeline RAG. La fase de indexación (superior) transforma los documentos en embeddings almacenados en una base de datos vectorial. La fase de recuperación (inferior) convierte la consulta del usuario en un embedding, recupera los fragmentos más relevantes y los proporciona como contexto al LLM para generar la respuesta.

3.2.2. Fase de indexación

La fase de indexación es el proceso previo mediante el cual los documentos originales se transforman en representaciones optimizadas para la búsqueda semántica. Este proceso consta de tres etapas: la carga de documentos, la fragmentación del texto y la generación de embeddings.

Carga y preprocesamiento de documentos

El primer paso consiste en extraer el contenido textual de las fuentes de información originales. Estas fuentes pueden ser de naturaleza muy diversa: archivos PDF, documentos de texto plano, hojas de cálculo, páginas web o contenido almacenado en la nube. Cada tipo de fuente requiere un procesador específico (*loader*) que se encargue de extraer el texto, limpiar artefactos de formato y normalizar el contenido para las etapas posteriores. La calidad de esta extracción inicial es determinante: errores en la carga —como la pérdida de tablas en documentos PDF o la mezcla de columnas en hojas de cálculo— se propagan inevitablemente a las fases de recuperación y generación.

Fragmentación del texto (*chunking*)

Una vez extraído el contenido textual, este debe dividirse en fragmentos (*chunks*) de tamaño manejable. La fragmentación es necesaria por dos razones principales: los modelos de embeddings tienen una longitud máxima de entrada (típicamente entre 256 y 8192 tokens), y los fragmentos más pequeños y específicos permiten una recuperación más precisa que los documentos completos (Gao et al., 2024b).

La estrategia de fragmentación tiene un impacto directo en la calidad del sistema. Si los fragmentos son demasiado grandes, contienen información heterogénea que diluye la

relevancia semántica; si son demasiado pequeños, pierden el contexto necesario para ser interpretados correctamente. Las estrategias más comunes incluyen:

- **Fragmentación por tamaño fijo:** se divide el texto en bloques de un número predeterminado de tokens o caracteres, generalmente con solapamiento (*overlap*) entre fragmentos consecutivos para evitar la pérdida de información en los límites de corte.
- **Fragmentación por estructura:** se utilizan las divisiones naturales del documento —párrafos, secciones, saltos de línea— como puntos de corte, preservando las unidades semánticas del texto original.
- **Fragmentación semántica:** se segmenta el texto en función de cambios en el significado, agrupando oraciones temáticamente relacionadas. Este enfoque es más sofisticado y costoso computacionalmente, pero produce fragmentos con mayor coherencia interna.

Un aspecto crítico de la fragmentación es la preservación de la relación entre cada fragmento y su documento de origen. Cada fragmento se almacena junto con **metadatos** que identifican el documento del que procede, su posición dentro del mismo, y cualquier información contextual relevante (título, autor, fecha, sección). Estos metadatos son esenciales para que, tras recuperar un fragmento relevante, el sistema pueda reconstruir el contexto completo —por ejemplo, accediendo al documento original o a los fragmentos adyacentes— cuando la respuesta lo requiera.

Generación de embeddings

Cada fragmento de texto se transforma en un vector numérico de alta dimensionalidad denominado *embedding*. Este vector captura el significado semántico del fragmento en un espacio geométrico donde textos con significados similares se sitúan en posiciones cercanas. La idea central de los embeddings, que tiene sus raíces en trabajos como Word2Vec (Mikolov et al., 2013), es que las relaciones semánticas entre conceptos pueden representarse como relaciones geométricas entre vectores: textos que tratan sobre temas similares producen vectores próximos entre sí, independientemente de las palabras exactas que utilicen.

Los modelos de embeddings actuales, como los basados en la arquitectura Sentence-BERT o los modelos comerciales de OpenAI y Cohere, generan vectores que capturan el significado a nivel de oración o párrafo, no solo a nivel de palabra. Esto permite que la búsqueda posterior sea verdaderamente semántica: la consulta «¿cuál es la política de devoluciones?» puede recuperar un fragmento que diga «los clientes disponen de 30 días para retornar cualquier producto», aunque no compartan prácticamente ninguna palabra en común (Karpukhin et al., 2020).

Los vectores resultantes se almacenan en una **base de datos vectorial** —como Pinecone, Weaviate, Qdrant o Chroma— optimizada para realizar búsquedas eficientes por

similitud en espacios de alta dimensionalidad. Cada entrada en la base de datos vectorial contiene el vector del embedding, el texto original del fragmento y los metadatos asociados, lo que permite tanto la recuperación semántica como la trazabilidad hacia el documento de origen.

3.2.3. Fase de recuperación y generación

Cuando un usuario formula una consulta, el sistema RAG ejecuta un proceso en tiempo real que consta de tres etapas: la transformación de la consulta, la recuperación de información relevante y la generación de la respuesta.

Búsqueda semántica sobre documentos vectorizados

La consulta del usuario se convierte en un embedding utilizando el mismo modelo empleado durante la indexación. Este vector de consulta se compara con los vectores almacenados en la base de datos vectorial mediante métricas de similitud —típicamente la similitud coseno o el producto escalar— para identificar los k fragmentos más relevantes. Este proceso, conocido como búsqueda por vecinos más cercanos (*k-nearest neighbors*, kNN), devuelve los fragmentos cuyo significado semántico es más afín a la pregunta del usuario, junto con una puntuación de similitud que indica el grado de relevancia (Karpukhin et al., 2020).

Es importante señalar que la búsqueda semántica trasciende la coincidencia léxica. Mientras que una búsqueda por palabras clave requeriría que la consulta y el documento compartan términos exactos, la búsqueda semántica opera sobre significados: identifica fragmentos relevantes incluso cuando se utilizan sinónimos, paráfrasis o formulaciones diferentes para expresar el mismo concepto. Esta propiedad es especialmente valiosa en aplicaciones de atención al cliente, donde los usuarios finales formulan sus consultas de formas muy diversas.

Existen también estrategias de **recuperación híbrida** que combinan la búsqueda semántica con técnicas léxicas clásicas (*sparse retrieval*, basadas en algoritmos como BM25 (Lin et al., 2021)) para que cada vía cubra las debilidades de la otra: la léxica brilla en consultas con términos exactos, códigos o nombres propios, mientras que la densa captura sinónimos y paráfrasis. Asimismo, los sistemas RAG avanzados pueden incorporar una fase posterior de **reordenamiento** (*re-ranking*) sobre los candidatos recuperados, típicamente con un *cross-encoder* basado en BERT (Nogueira and Cho, 2019), para refinar la relevancia antes de pasar el contexto al LLM. Estas técnicas no se incorporan al MVP de la plataforma propuesta y quedan como extensiones naturales del puerto de recuperación.

Acceso a datos estructurados mediante text-to-SQL

No toda la información relevante para un chatbot reside en documentos de texto. Muchas organizaciones almacenan datos críticos —catálogos de productos, registros de clientes, inventarios, historiales de transacciones— en bases de datos relacionales (SQL).

Para estos casos, la búsqueda semántica sobre documentos vectorizados resulta insuficiente: la información está estructurada en tablas con relaciones definidas, y las consultas del usuario requieren operaciones como filtrado, agregación o unión de datos.

En este escenario, el sistema utiliza una estrategia diferente: en lugar de buscar fragmentos similares, analiza la consulta del usuario junto con el **esquema de la base de datos** (nombres de tablas, columnas y sus tipos) para generar una consulta SQL que extraiga la información necesaria. El LLM actúa aquí como traductor de lenguaje natural a SQL (*text-to-SQL*), una capacidad que los modelos actuales han demostrado con notable eficacia (Gao et al., 2024a). Por ejemplo, ante la pregunta «¿cuántos pedidos se realizaron en marzo?», el sistema genera y ejecuta una consulta como `SELECT COUNT(*) FROM pedidos WHERE mes = 3`, y utiliza el resultado como contexto para la respuesta.

Permitir que un LLM genere y ejecute consultas SQL contra una base de datos productiva introduce riesgos de seguridad evidentes: tanto una inyección maliciosa formulada en lenguaje natural como una generación errónea pueden traducirse en operaciones destructivas (`DROP`, `DELETE`, `UPDATE`) o en accesos no autorizados a información sensible (Pedro et al., 2025). Para mitigarlo, los sistemas *text-to-SQL* en producción aplican defensa en profundidad mediante varias capas complementarias:

- **Principio de mínimo privilegio en la conexión:** el sistema accede a la base de datos con un usuario de base de datos que dispone únicamente de permisos de `SELECT`, restringidos además a las tablas y columnas que el administrador haya autorizado. Cualquier intento de operación que exceda esos permisos es rechazado por el propio motor de la base de datos.
- **Validación sintáctica de la consulta generada:** antes de la ejecución, un *parser* analiza la sentencia SQL producida por el LLM y bloquea las que contengan instrucciones de modificación o definición (DML/DDL más allá de `SELECT`), comentarios sospechosos o múltiples sentencias encadenadas.
- **Revisión semántica con un LLM ligero:** una capa adicional emplea un modelo rápido y barato cuya única tarea es clasificar la consulta SQL generada como benigna o sospechosa antes de su ejecución, capturando intentos de exfiltración o evasión que pasan los filtros sintácticos.

Estas tres capas son complementarias: la primera limita el daño posible incluso si las otras fallan, la segunda detiene la mayoría de consultas peligrosas con coste mínimo, y la tercera aporta una red de seguridad adicional para entornos especialmente sensibles.

Enrutamiento de consultas multi-fuente

La coexistencia de recuperación semántica sobre documentos y consulta estructurada sobre bases de datos motiva la necesidad de un **sistema de enrutamiento** (*query routing*) que, ante cada consulta del usuario, determine qué estrategia de recuperación es la más apropiada —o si debe combinar ambas— en función de la naturaleza de la pregunta

y de las fuentes de conocimiento disponibles (Gao et al., 2024b). El enrutador puede implementarse mediante reglas explícitas (clasificación basada en palabras clave o patrones de la consulta), mediante un clasificador entrenado, o delegando la decisión al propio LLM con un *prompt* específico que pondere las características de la consulta y las fuentes disponibles. La Figura 3.3 ilustra este mecanismo.

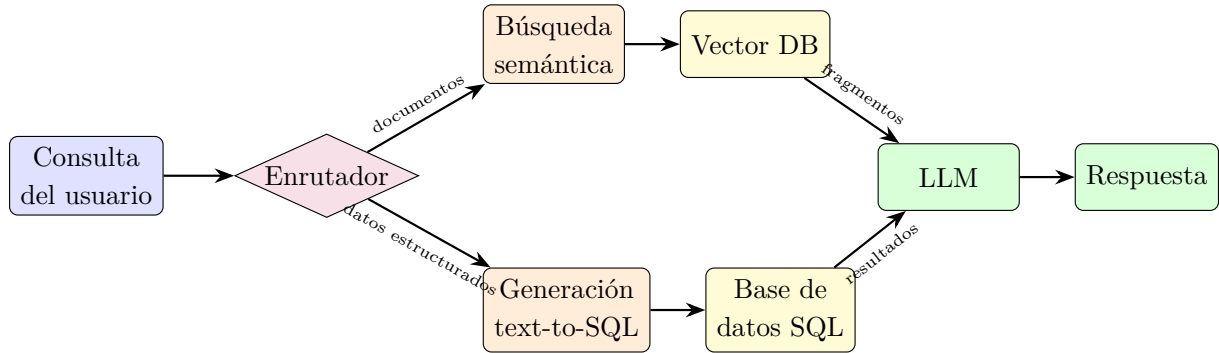


Figura 3.3: Enrutamiento de consultas en un sistema RAG multi-fuente. El enrutador analiza la consulta y la dirige hacia la estrategia de recuperación apropiada: búsqueda semántica para documentos no estructurados o generación de SQL para datos tabulares. Los resultados de ambas vías se proporcionan como contexto al LLM.

Generación aumentada de la respuesta

Una vez recuperada la información relevante —ya sean fragmentos de documentos, resultados de consultas SQL o una combinación de ambos— el sistema construye un *prompt* enriquecido que se envía al modelo de lenguaje. Este *prompt* combina tres elementos:

1. **Instrucciones del sistema:** las directivas que definen el rol y comportamiento del chatbot, configuradas por el administrador (por ejemplo, «Eres un asistente de soporte técnico. Responde únicamente con información de los documentos proporcionados»).
2. **Contexto recuperado:** los fragmentos de texto o datos estructurados obtenidos en la fase de recuperación, que proporcionan al modelo la información factual necesaria para responder.
3. **Consulta del usuario:** la pregunta original tal como la formuló el usuario.

El LLM genera entonces una respuesta que sintetiza la información recuperada en lenguaje natural, adaptándola al tono y formato definidos en las instrucciones del sistema. Al disponer de evidencia concreta en la que fundamentar su respuesta, el modelo reduce significativamente la tendencia a alucinar: en lugar de recurrir a su conocimiento paramétrico (potencialmente incorrecto o desactualizado), se apoya en los datos proporcionados (Lewis et al., 2020). Además, dado que los fragmentos recuperados son identificables y trazables, el sistema puede citar las fuentes utilizadas, proporcionando al usuario final la transparencia que los LLMs por sí solos no ofrecen.

3.2.4. Variantes y evolución

Desde su formulación original, el paradigma RAG ha evolucionado de forma significativa. La propuesta inicial de Lewis et al. (2020) planteaba RAG como una arquitectura entrenada de extremo a extremo, en la que un *retriever* denso (DPR) y un generador secuencia a secuencia (BART) se optimizaban conjuntamente sobre datos etiquetados. La práctica industrial actual ha derivado hacia un enfoque más pragmático en el que el *retriever* y el LLM se mantienen como componentes independientes preentrenados, y la integración entre ambos se reduce a inyectar los fragmentos recuperados en el contexto del LLM mediante *prompting*. Esta separación, aunque sacrifica la optimización conjunta, simplifica drásticamente el despliegue y permite combinar cualquier *retriever* con cualquier LLM sin reentrenamiento. Es la aproximación que sigue la plataforma propuesta. Gao et al. (2024b) identifican tres niveles de madurez en las implementaciones que aplican este enfoque pragmático:

- **RAG ingenuo** (*Naive RAG*): sigue el flujo lineal descrito anteriormente —indexar, recuperar, generar— sin optimizaciones adicionales. Aunque funcional, presenta limitaciones como la recuperación de fragmentos irrelevantes, la redundancia en el contexto y la dificultad para manejar consultas complejas que requieren razonamiento multi-paso.
- **RAG avanzado** (*Advanced RAG*): introduce mejoras en las fases de pre-recuperación y post-recuperación. En la pre-recuperación, se aplican técnicas como la reescritura de consultas (*query rewriting*), la descomposición de preguntas complejas o la expansión de consultas para mejorar la cobertura de la búsqueda. En la post-recuperación, se reordenan (*reranking*) los fragmentos recuperados para priorizar los más relevantes y se comprimen para eliminar información redundante antes de enviarlos al modelo generativo.
- **RAG modular** (*Modular RAG*): descompone el pipeline en módulos intercambiables —enrutamiento, recuperación, generación— que pueden combinarse y configurarse de forma flexible según el caso de uso. Esta es la **categoría arquitectónica** que adopta la plataforma propuesta: cada componente del pipeline es intercambiable a través de los puertos del dominio (Sección 3.4). El motor RAG concreto que implementa el MVP corresponde, en términos funcionales, a un **RAG avanzado** con enrutamiento de consultas; las técnicas adicionales de pre-recuperación y post-recuperación propias de esa categoría (reescritura de consultas, reordenación, re-recuperación) se identifican como módulos opcionales que el carácter modular permite incorporar sin tocar el núcleo.

3.2.5. Limitaciones y desafíos

A pesar de sus ventajas, los sistemas RAG presentan desafíos que condicionan su implementación práctica. Estos pueden agruparse en dos categorías: **limitaciones in-**

trínsecas al pipeline, derivadas del propio funcionamiento del paradigma, y **desafíos transversales**, que afectan al despliegue y a la operación del sistema en contextos reales.

Limitaciones intrínsecas al pipeline.

- **Calidad de la recuperación:** si los fragmentos recuperados no son relevantes para la consulta, el modelo puede generar respuestas incorrectas fundamentadas en contexto inadecuado —un problema a veces más perjudicial que la propia alucinación, ya que la respuesta aparenta estar respaldada por evidencia (Wu et al., 2024).
- **Dependencia de la estrategia de fragmentación:** la forma en que se dividen los documentos influye directamente en qué información es recuperable. Una fragmentación inadecuada puede separar datos que deberían consultarse juntos o agrupar información heterogénea que diluye la relevancia semántica.
- **Ventana de contexto limitada:** aunque los LLMs actuales admiten ventanas de contexto cada vez mayores, incluir demasiados fragmentos puede degradar la calidad de la respuesta. Liu et al. (2024) demostraron empíricamente el fenómeno conocido como *lost in the middle*: los modelos atienden con mucha más eficacia a la información situada al principio o al final del contexto que a la situada en posiciones intermedias, lo que implica que el orden y la cantidad de fragmentos proporcionados influyen directamente en la calidad de la respuesta (Gao et al., 2024b).
- **Consultas que requieren razonamiento multi-fuente:** cuando la respuesta a una pregunta requiere combinar información de varios documentos o fuentes heterogéneas (por ejemplo, datos de un PDF y registros de una base de datos SQL), el sistema debe ser capaz de orquestar múltiples recuperaciones y sintetizar la información, una tarea que las implementaciones básicas no resuelven (Chen et al., 2020; Talmor et al., 2021).

Desafíos transversales.

- **Evaluación:** medir la calidad de un sistema RAG es más complejo que evaluar un modelo generativo aislado, ya que requiere valorar tanto la relevancia de lo recuperado como la fidelidad y utilidad de la respuesta generada. Esta dualidad motiva la necesidad de marcos de evaluación específicos como RAGAS, que se abordará en el Capítulo 4.
- **Privacidad y cumplimiento normativo:** los sistemas RAG ingieren y vectorizan información a menudo sensible (documentación interna, datos de clientes, registros corporativos), lo que los sitúa de lleno en el ámbito de regulaciones como el Reglamento General de Protección de Datos (RGPD) europeo. El uso de proveedores externos de modelos de lenguaje y embeddings implica que esos datos viajan a infraestructuras de terceros, lo que exige acuerdos de tratamiento de datos, mecanismos

de anonimización o, alternativamente, despliegues completamente autoalojados. Esta tensión entre capacidad y soberanía del dato es uno de los motores principales del interés en modelos locales y plataformas autoalojables como la propuesta en este trabajo.

- **Coste y latencia agregados:** a diferencia de una llamada directa al LLM, una consulta RAG involucra al menos tres operaciones secuenciales —generación del *embedding* de la consulta, búsqueda en la base de datos vectorial y llamada al LLM con el contexto recuperado— a las que pueden sumarse fases de reordenamiento o consultas SQL. El coste y la latencia totales de una respuesta RAG son, por tanto, sensiblemente mayores que los de una llamada simple al LLM, lo que obliga a optimizar cada etapa y a aplicar técnicas de *caching* cuando el caso de uso lo permite.

Estos desafíos justifican las decisiones de diseño de la plataforma propuesta, particularmente la necesidad de ofrecer configurabilidad en las estrategias de fragmentación, permitir la combinación de fuentes heterogéneas y proporcionar un sistema de enrutamiento inteligente que seleccione la estrategia de recuperación adecuada para cada consulta.

3.3. Embeddings y bases de datos vectoriales

3.3.1. Definición y fundamentos

Un *embedding* es una función que proyecta un elemento discreto —una palabra, oración, párrafo o documento— en un vector de números reales en un espacio de dimensionalidad fija (típicamente entre 256 y 4096 dimensiones). La propiedad fundamental de esta proyección es que **preserva las relaciones semánticas**: elementos con significados similares se representan como vectores cercanos en el espacio, mientras que elementos semánticamente distintos se sitúan en regiones alejadas (Mikolov et al., 2013).

Los primeros modelos de embeddings, como Word2Vec (Mikolov et al., 2013), operaban a nivel de palabra individual y asignaban un único vector a cada término del vocabulario. Esto implicaba que una palabra polisémica como «banco» (entidad financiera o asiento) recibía la misma representación independientemente del contexto. Los modelos contextuales posteriores, basados en la arquitectura Transformer, resolvieron esta limitación: BERT (Devlin et al., 2019) y sus derivados generan representaciones que varían según el contexto en el que aparece cada palabra, capturando matices semánticos que los modelos estáticos no podían distinguir.

3.3.2. Embeddings a nivel de oración y documento

Para los sistemas RAG, el nivel relevante de representación no es la palabra individual sino el fragmento de texto completo. Los modelos de embeddings a nivel de oración, como Sentence-BERT (Reimers and Gurevych, 2019), adaptan la arquitectura de BERT para

producir un único vector que represente el significado global de una oración o párrafo. La técnica consiste en procesar el texto a través del modelo y aplicar una operación de agrupación (*pooling*) —típicamente el promedio de los vectores de todos los tokens— para obtener una representación de dimensionalidad fija.

Esta representación unificada es la que permite la búsqueda semántica descrita en la sección anterior: al codificar tanto los fragmentos de la base de conocimiento como la consulta del usuario en el mismo espacio vectorial, la similitud entre sus vectores refleja la afinidad semántica entre sus significados. Los modelos de embeddings comerciales actuales —como los ofrecidos por OpenAI— y los modelos de código abierto —como los de las familias E5 o BGE— están específicamente entrenados para maximizar esta propiedad, utilizando técnicas de aprendizaje contrastivo (Gao et al., 2021) que acercan los vectores de textos relacionados y alejan los de textos no relacionados.

Junto a estos **embeddings densos** existen alternativas exploradas en el estado del arte de la recuperación, como la *late interaction* (ColBERT (Khattab and Zaharia, 2020)) o los **embeddings dispersos** (SPLADE (Formal et al., 2021)), que quedan fuera del alcance del MVP de la plataforma propuesta.

3.3.3. Métricas de similitud

La búsqueda en el espacio vectorial requiere una métrica que cuantifique la cercanía entre vectores. Las dos métricas más utilizadas en sistemas RAG son:

- **Similitud coseno:** mide el coseno del ángulo entre dos vectores, produciendo un valor entre -1 (opuestos) y 1 (idénticos). Al depender únicamente de la dirección y no de la magnitud, es robusta frente a diferencias en la longitud del texto:

$$\text{sim}(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|}$$

- **Producto escalar** (*dot product*): equivalente a la similitud coseno cuando los vectores están normalizados (norma unitaria), que es el caso habitual en los modelos de embeddings modernos. Es computacionalmente más eficiente al evitar el cálculo de las normas.

3.3.4. Bases de datos vectoriales

Las bases de datos tradicionales —tanto relacionales (SQL) como documentales (NoSQL)— están optimizadas para búsquedas exactas: localizar registros que coincidan con un valor concreto, cumplan una condición o se encuentren en un rango. Sin embargo, los embeddings requieren un tipo de consulta fundamentalmente diferente: dada una representación vectorial, encontrar los k vectores más cercanos entre millones o miles de millones de registros. Este problema, conocido como búsqueda de vecinos más cercanos (*Nearest Neighbor Search*), no puede resolverse eficientemente con índices convencionales como los árboles B o las tablas hash (Johnson et al., 2021).

Las **bases de datos vectoriales** son sistemas de almacenamiento especializados en este tipo de consultas. A diferencia de una base de datos relacional, donde la unidad de almacenamiento es una fila con columnas tipadas, en una base de datos vectorial la unidad fundamental es un **registro vectorial** compuesto por:

- El **vector de embedding**: la representación numérica del fragmento de texto.
- El **contenido original**: el texto del fragmento tal como fue indexado.
- Los **metadatos**: información adicional como el documento de origen, la posición dentro del mismo, la fecha de ingestión u otros campos definidos por el administrador.

Esta estructura permite que, una vez recuperado un fragmento relevante mediante búsqueda vectorial, el sistema pueda acceder tanto al texto original (para proporcionarlo como contexto al LLM) como a los metadatos (para trazar la procedencia de la información o acceder al documento completo si es necesario).

3.3.5. Algoritmos de indexación y búsqueda aproximada

La búsqueda exacta de vecinos más cercanos requiere comparar el vector de consulta con todos los vectores almacenados, lo que resulta computacionalmente prohibitivo para colecciones grandes (la complejidad crece linealmente con el número de vectores). Para resolver este problema, las bases de datos vectoriales emplean algoritmos de **búsqueda aproximada de vecinos más cercanos** (*Approximate Nearest Neighbor*, ANN), que sacrifican una pequeña fracción de precisión a cambio de reducciones drásticas en el tiempo de búsqueda (Malkov and Yashunin, 2020).

El algoritmo dominante en bases de datos vectoriales modernas es **HNSW** (*Hierarchical Navigable Small World*) (Malkov and Yashunin, 2020): construye un grafo multicapa donde cada nodo es un vector, y la búsqueda navega desde las capas superiores —con conexiones de largo alcance— hacia las inferiores —con conexiones locales—, convergiendo rápidamente hacia los vecinos más cercanos. Ofrece un equilibrio favorable entre velocidad y precisión, motivo por el que Qdrant lo emplea por defecto. Existen también familias alternativas basadas en particionado del espacio (**IVF**, *Inverted File Index*) y en cuantización de los vectores (*Product Quantization* (Jégou et al., 2011)), útiles especialmente en colecciones masivas o con restricciones de memoria (Johnson et al., 2021); las bases de datos vectoriales modernas abstraen estas complejidades, por lo que la plataforma propuesta no expone su selección al administrador en la versión actual.

La elección y configuración de los embeddings y la base de datos vectorial son decisiones críticas en cualquier sistema RAG, ya que determinan tanto la calidad de la recuperación como el rendimiento del sistema. En la plataforma propuesta, estas decisiones se exponen como parámetros configurables: el administrador puede seleccionar el modelo de embeddings, ajustar la dimensionalidad de los vectores y configurar los parámetros de búsqueda (número de resultados k , umbral de similitud mínima). La arquitectura

de puertos y adaptadores, que se detalla en la siguiente sección, permite intercambiar el proveedor de embeddings o la base de datos vectorial sin modificar la lógica del sistema.

3.4. Arquitectura de software modular

3.4.1. Definición y fundamentos

La arquitectura de software define la estructura de alto nivel de un sistema: cómo se organizan sus componentes, cómo se comunican entre sí y qué principios rigen sus dependencias. En sistemas con requisitos evolutivos —donde los proveedores externos pueden cambiar, las fuentes de datos pueden ampliarse y los componentes deben poder sustituirse sin afectar al resto— la modularidad arquitectónica se convierte en un requisito fundamental ([Martin, 2018](#)).

Una arquitectura modular organiza el sistema en módulos con responsabilidades bien definidas y fronteras claras, de modo que cada módulo pueda desarrollarse, probarse y modificarse de forma independiente. El principio rector es la **inversión de dependencias**: los módulos de alto nivel (lógica de negocio) no dependen de los módulos de bajo nivel (infraestructura), sino que ambos dependen de abstracciones. Esto permite que los detalles de implementación —qué base de datos se usa, qué API se consulta, qué modelo de lenguaje se invoca— sean intercambiables sin modificar la lógica central del sistema ([Martin, 2018](#)).

3.4.2. El patrón de puertos y adaptadores

El patrón de **puertos y adaptadores** (*ports and adapters*), también conocido como *arquitectura hexagonal*, fue propuesto por [Cockburn \(2005\)](#) como respuesta a un problema recurrente en el desarrollo de software: la tendencia a que la lógica de negocio se acople progresivamente a los detalles de infraestructura (bases de datos, interfaces de usuario, servicios externos), dificultando su evolución y testabilidad.

La arquitectura se organiza en tres capas concéntricas:

- **Dominio:** el núcleo del sistema, que contiene las reglas de negocio y las entidades del problema. No tiene dependencias hacia el exterior: no conoce bases de datos, frameworks ni servicios externos. En el contexto de la plataforma propuesta, el dominio modela conceptos como *chatbot*, *fuentes de conocimiento*, *conversación* o *fragmento*.
- **Puertos:** interfaces (contratos) que definen cómo el dominio se comunica con el mundo exterior. Existen puertos de entrada (*driving ports*), que representan las operaciones que el sistema ofrece (por ejemplo, «crear un chatbot» o «procesar una consulta»), y puertos de salida (*driven ports*), que representan las capacidades que el dominio necesita del exterior (por ejemplo, «almacenar un vector» o «generar texto con un LLM»).

- **Adaptadores:** implementaciones concretas de los puertos que conectan el dominio con tecnologías específicas. Un adaptador de salida para el puerto «almacenar un vector» podría implementarse con Pinecone, Qdrant o Chroma; un adaptador para «generar texto» podría conectar con OpenAI, Anthropic o un modelo local. Sustituir un proveedor se reduce a implementar un nuevo adaptador que cumpla el contrato del puerto, sin modificar el dominio ni los demás adaptadores (Cockburn, 2005).

Este patrón es especialmente adecuado para la plataforma propuesta por dos razones. En primer lugar, el ecosistema de modelos de lenguaje, embeddings y bases de datos vectoriales evoluciona con rapidez: nuevos proveedores aparecen, los modelos existentes se deprecian y los precios cambian; la arquitectura hexagonal permite absorber estos cambios sin reestructurar el sistema. En segundo lugar, la plataforma debe soportar múltiples proveedores simultáneamente (el administrador selecciona qué modelo y qué base vectorial usa cada chatbot), lo que permite además que distintos clientes desplegados sobre la misma instancia operen con proveedores distintos según sus restricciones de coste, soberanía del dato o preferencias técnicas, y exige que las implementaciones sean intercambiables en tiempo de ejecución.

A esta motivación funcional se suma un beneficio metodológico: la **testabilidad** del sistema. Al depender el dominio exclusivamente de interfaces abstractas, las implementaciones concretas pueden sustituirse trivialmente por dobles de prueba (*mocks* o *fakes*) durante el desarrollo y la verificación. Las pruebas unitarias del dominio pueden ejecutarse sin conexión a OpenAI, sin instancia de Qdrant y sin acceso a una base de datos PostgreSQL real, lo que reduce drásticamente el coste y la fragilidad de la batería de pruebas. Esta propiedad se aprovecha en la estrategia de pruebas descrita en el Capítulo 4.

La Figura 3.4 ilustra la aplicación concreta del patrón a la plataforma. El dominio expone dos puertos de entrada (*driving*) —**AdminAPI** (operaciones del panel: crear y configurar chatbots) y **ChatAPI** (mensajes del usuario final)— y declara los puertos de salida (*driven*) que necesita para operar: **LLMProvider** (modelo de lenguaje), **EmbeddingsProvider** (modelo de embeddings), **VectorStore** (base de datos vectorial), **DocumentLoader** (carga de documentos), **QueryRouter** (enrutador de consultas) y **SQLDataSource** (acceso a bases de datos relacionales). Cada puerto es una interfaz que define un contrato; cada adaptador es una implementación concreta independiente. Sustituir un proveedor —cambiar Qdrant por otra base vectorial, o el LLM de OpenAI por un modelo Ollama local— se reduce a implementar un nuevo adaptador que cumpla el contrato del puerto correspondiente, sin tocar el dominio ni el resto de adaptadores.

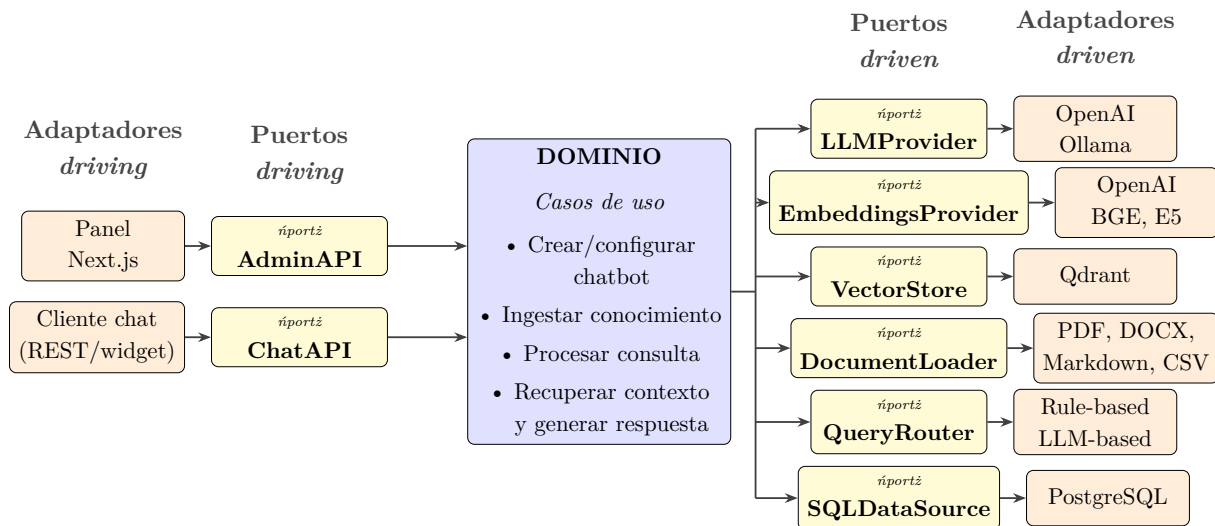


Figura 3.4: Aplicación del patrón de puertos y adaptadores a la plataforma. Elaboración propia basada en Cockburn (2005).

A modo de ejemplo concreto, el puerto `LLMProvider` se materializa en el código como una interfaz abstracta con un único método: `generate(prompt: str) -> str`. Los adaptadores `OpenAILLMAdapter` y `OllamaLLMAdapter` implementan esa interfaz, cada uno encapsulando la lógica específica de invocación a su servicio (gestión de claves API, formato de petición, parseo de respuesta). El dominio recibe la implementación correspondiente por inyección de dependencias en función de la configuración del chatbot e invoca `llm.generate(prompt)` sin distinguir cuál de las dos está activa. Añadir soporte para un nuevo proveedor —por ejemplo, Anthropic— consiste en escribir un `AnthropicLLMAdapter` que cumpla el mismo contrato y registrarlo en el sistema de configuración; el dominio y el resto de adaptadores permanecen intactos.

Conviene situar el patrón frente a alternativas relacionadas. La **arquitectura por capas** tradicional (*Layered Architecture*) organiza el sistema en capas horizontales —presentación, lógica, datos— con dependencias dirigidas hacia abajo, pero permite que la lógica de negocio dependa de la capa de datos, lo que en la práctica acopla el dominio a tecnologías concretas. Existen formulaciones afines al patrón hexagonal —la **Onion Architecture** (Palermo, 2008) y la **Clean Architecture** (Martin, 2018)— que comparten el principio de inversión de dependencias proponiendo capas concéntricas con la lógica de negocio en el centro. Se adopta la formulación de Cockburn por la distinción explícita entre puertos *driving* y *driven*, que se ajusta naturalmente a un sistema con dos modos de invocación heterogéneos —panel administrativo y consultas del usuario final del chatbot—.

La adopción de este patrón conlleva costes que conviene reconocer. La indirección entre dominio y adaptadores aumenta la complejidad inicial del código y la curva de aprendizaje para nuevos desarrolladores: cada capacidad del sistema requiere al menos una interfaz y una implementación, frente a la invocación directa que permitiría una arquitectura más sencilla. La inyección de dependencias añade además una pequeña indirección en tiempo de ejecución, irrelevante en un sistema dominado por la latencia de las llamadas a LLMs

y bases de datos vectoriales, pero que conviene tener presente. Estos costes son asumibles en el presente trabajo precisamente porque la modularidad no es un medio sino un fin: la plataforma se diseña para que sus componentes sean intercambiables por terceros, lo que justifica la infraestructura adicional que el patrón impone.

3.4.3. Patrones complementarios

La arquitectura hexagonal se complementa con otros patrones que refinan la organización interna del sistema:

- **Patrón estrategia** (*Strategy*): permite definir una familia de algoritmos intercambiables encapsulados tras una interfaz común (Gamma et al., 1994). En la plataforma, este patrón se aplica a las estrategias de fragmentación (tamaño fijo, por estructura, semántica) y de enrutamiento de consultas: el administrador selecciona la estrategia deseada, y el sistema la instancia sin que la lógica de orquestación cambie.
- **Inyección de dependencias** (*Dependency Injection*): mecanismo mediante el cual los adaptadores concretos se proporcionan al dominio en tiempo de ejecución, en lugar de ser instanciados directamente por él. Esto materializa la inversión de dependencias y facilita tanto la testabilidad (sustituyendo adaptadores reales por dobles de prueba) como la configurabilidad (seleccionando adaptadores según la configuración de cada chatbot).

3.4.4. Conexión con la implementación

La materialización concreta de esta arquitectura —el código fuente de la plataforma, los adaptadores efectivamente desarrollados en el MVP, la organización del proyecto en módulos y paquetes Python, el cableado de la inyección de dependencias en función de la configuración de cada chatbot y el flujo desde el panel de administración hasta los puertos del dominio— se documenta en el Capítulo 5. Allí el patrón teórico aquí descrito se traduce en estructura real de proyecto, con las decisiones de diseño detalladas y justificadas a la luz de los principios introducidos en esta sección.

3.5. Estado del arte y trabajos relacionados

3.5.1. Frameworks de orquestación RAG

El desarrollo de aplicaciones basadas en RAG se ha visto facilitado por la aparición de frameworks de orquestación que proporcionan abstracciones y componentes reutilizables para construir pipelines de recuperación y generación.

LangChain (Chase, 2022) es uno de los frameworks de orquestación de aplicaciones LLM con mayor adopción. Ofrece un ecosistema de módulos para la carga de documentos, fragmentación, generación de embeddings, almacenamiento vectorial y orquestación

de cadenas de procesamiento, con una amplia variedad de integraciones que permiten conectar prácticamente cualquier combinación de proveedores. Sin embargo, LangChain es una biblioteca de desarrollo: construir una aplicación funcional requiere escribir código, diseñar la arquitectura y gestionar el despliegue, lo que lo sitúa como una herramienta para desarrolladores, no para administradores funcionales.

LlamaIndex (LlamaIndex, 2023) se centra específicamente en la conexión entre fuentes de datos y modelos de lenguaje. A diferencia de LangChain, que aborda el problema de forma generalista, LlamaIndex optimiza la indexación y recuperación de datos, ofreciendo abstracciones de alto nivel para la ingestión multi-fuente y la construcción de índices. Al igual que LangChain, es un framework programático dirigido a desarrolladores.

Haystack (deepset, 2024), desarrollado por deepset, es un framework open-source orientado específicamente a sistemas de búsqueda semántica y RAG en entornos productivos. A diferencia de LangChain, que abarca un espectro más amplio de aplicaciones LLM, Haystack se centra en pipelines de recuperación y generación con énfasis en la fiabilidad y la trazabilidad: incorpora componentes nativos para evaluación, monitorización y despliegue, y ofrece un modelo de *pipeline* declarativo que facilita su uso en entornos empresariales. Comparte con los anteriores su naturaleza programática: construir una aplicación funcional requiere desarrollar código.

Estos frameworks resuelven el problema técnico de la orquestación RAG, pero no abordan la necesidad de una interfaz accesible para administradores sin conocimientos de programación. La plataforma propuesta en este trabajo utiliza conceptos y patrones similares a los de estos frameworks, pero los encapsula tras un panel de administración web que permite configurar el pipeline completo sin escribir código.

3.5.2. Plataformas comerciales de chatbots

El mercado de plataformas de chatbot ha evolucionado significativamente con la adopción de LLMs. Las soluciones comerciales actuales —Dialogflow CX (Google), Amazon Lex (AWS), Azure Bot Service (Microsoft), Voiceflow, CustomGPT, Chatbase— cubren un espectro amplio de funcionalidades: desde flujos conversacionales dirigidos hasta capacidades RAG integradas con soporte para múltiples proveedores de LLM. Plataformas como Dialogflow, por ejemplo, son agnósticas respecto al modelo de lenguaje y ofrecen integraciones con fuentes de datos diversas. En rigor, la mayoría podrían, con la configuración y el desarrollo adecuados, reproducir las funcionalidades que propone este trabajo.

Sin embargo, estas soluciones comparten varias características que las diferencian de la plataforma propuesta. Son arquitectónicamente cerradas y propietarias: el administrador consume los componentes tal como el proveedor los ofrece, sin posibilidad de sustituir un módulo concreto —por ejemplo, el sistema de embeddings o la base de datos vectorial— por una implementación propia. Generan dependencia del proveedor y suelen restringir las opciones avanzadas (modelos locales, almacenamiento autoalojado) a los planes de pago superiores. Y aunque algunas permiten seleccionar entre varios modelos o fuentes de datos, los módulos del pipeline no están genuinamente desacoplados: el administrador

puede elegir entre las opciones del menú, pero no desconectar un módulo y sustituirlo por una implementación alojada en su propia infraestructura.

3.5.3. Plataformas open-source de bajo código

Junto a las soluciones comerciales, en los últimos años ha emergido una categoría intermedia de plataformas **open-source y autoalojables** que incorporan interfaces visuales de bajo código para construir aplicaciones basadas en LLM y RAG. Las más representativas son:

- **Dify** ([LangGenius, Inc., 2024](#)): plataforma open-source orientada específicamente a aplicaciones LLM y RAG. Combina un orquestador de modelos con un editor visual de flujos y un módulo de despliegue como API o widget de chat. Soporta múltiples proveedores de LLM, varias bases vectoriales y dispone de un sistema de plugins que permite extender la funcionalidad.
- **Flowise** ([FlowiseAI, 2024](#)) y **Langflow** ([Langflow, 2024](#)): constructores visuales de flujos basados en componentes arrastrables que se apoyan, respectivamente, en LangChain y en su ecosistema. Permiten encadenar bloques de carga de documentos, fragmentación, recuperación y generación, exportando el resultado como API consumible.
- **Botpress** ([Botpress Technologies Inc., 2024](#)): una de las plataformas open-source de chatbot más extendidas, con licencia AGPLv3 y un ecosistema maduro. Recientemente ha incorporado capacidades RAG sobre fuentes documentales con un panel visual.

Estas plataformas reducen significativamente la barrera de entrada y, en términos funcionales, cubren un perímetro similar al que se propone en este trabajo: son open-source, autoalojables y permiten configurar pipelines RAG sin programar. Su modularidad, sin embargo, se materializa principalmente como **selección entre proveedores soportados** o como **sistema de plugins informales**, no como una arquitectura formal de puertos y adaptadores documentada en los términos de la sección 3.4. Tampoco abordan de forma nativa la integración de fuentes documentales y bases SQL bajo un módulo de enrutamiento explícito ni los mecanismos de seguridad por capas para *text-to-SQL* descritos anteriormente.

3.5.4. Avances recientes en el paradigma RAG

La investigación académica ha propuesto diversas mejoras sobre el paradigma RAG original que extienden el motor de recuperación y generación en cuatro direcciones complementarias: la **autorreflexión** sobre la necesidad y la calidad de la recuperación (**Self-RAG** ([Asai et al., 2024](#))); la **recuperación activa** en la que el modelo decide dinámicamente cuándo consultar fuentes externas durante la generación (**Active RAG** ([Jiang](#)

et al., 2023)); la **reformulación previa de la consulta** mediante una respuesta hipotética cuyo embedding sustituye al de la consulta original (**HyDE** (Gao et al., 2023)); y la **construcción previa de un grafo de conocimiento** sobre el corpus que captura entidades y relaciones para responder a preguntas que requieren síntesis global o razonamiento multi-documento (**GraphRAG** (Edge et al., 2024)).

El motor RAG del MVP de la plataforma propuesta corresponde a una implementación de RAG avanzado con enrutamiento de consultas y no incorpora estas técnicas. Su carácter modular —el contrato de los puertos del dominio, descrito en la Sección 3.4— las habilita como módulos opcionales en iteraciones futuras sin tocar el núcleo del sistema.

3.5.5. Avances recientes en modelos de embeddings

La calidad de los embeddings condiciona directamente la calidad de la recuperación en cualquier sistema RAG. La evolución reciente apunta en tres direcciones complementarias relevantes para la plataforma propuesta:

- **Embeddings de alta calidad contruidos sobre modelos generativos**, que aprovechan la representación interna de un LLM para producir vectores de texto (ejemplo reciente: Gemini Embedding 2 (Google, 2026)).
- **Embeddings multimodales unificados**, que proyectan modalidades distintas —texto, imagen, vídeo, audio y documentos como PDFs— en un único espacio vectorial y permiten búsquedas cruzadas entre tipos de archivo. Esta dirección es especialmente relevante para la plataforma porque amplía el tipo de fuentes de conocimiento explotables más allá del texto plano.
- **Embeddings open-source ligeros para ejecución local** (EmbeddingGemma (Google, 2025), las familias E5 (Wang et al., 2024) y BGE (Xiao et al., 2024)), que permiten desplegar un sistema RAG completo sin enviar datos sensibles a APIs externas.

La arquitectura modular de la plataforma permite incorporar cualquiera de estas familias mediante un adaptador que cumpla el contrato del puerto de embeddings. El MVP del presente trabajo se centra en embeddings de texto; la incorporación de embeddings multimodales se identifica como línea de extensión natural del sistema.

3.5.6. Posicionamiento del presente trabajo

Conviene distinguir dos perfiles de uso al hablar de la plataforma propuesta —perfiles de la plataforma misma, no del chatbot que ésta despliega; el usuario final del chatbot es un tercer actor, externo a esta clasificación—. El **administrador funcional** —sin conocimientos de programación— crea y configura chatbots desde el panel web combinando bloques predefinidos: a sus ojos, la plataforma es una herramienta No-Code. El **desarrollador de extensiones** —perfil técnico, equipo interno o tercero— puede añadir

nuevos adaptadores (un proveedor de LLM no soportado, una base vectorial específica, una estrategia de fragmentación a medida) sin tocar el núcleo del sistema: a sus ojos, la plataforma es una arquitectura modular extensible. La arquitectura hexagonal descrita en la Sección 3.4 es el habilitador técnico de esta dualidad: las decisiones del panel web se traducen en la selección y configuración de adaptadores concretos, mientras que los puertos del dominio quedan abiertos a nuevas implementaciones que el desarrollador puede ejecutar incluso en su propia infraestructura —por privacidad, por cumplimiento normativo o por adaptación a un dominio específico—.

La Tabla 3.1 resume las diferencias entre las soluciones existentes y la plataforma propuesta a lo largo de los ejes anteriores.

Cuadro 3.1: Comparativa entre soluciones existentes y la plataforma propuesta.

Característica	Frameworks (LangChain, LlamaIndex, Haystack)	Plataformas comerciales (Dialogflow)	Open-source low-code (Dify, Flowise)	Plataforma propuesta
No-Code para administrador	No	Sí	Sí	Sí
Open-source y autoalojable	Sí	Parcial	Sí	Sí
Soporte de modelos LLM locales sin restricciones	Sí (manual)	Limitado	Sí	Sí
Sustitución por contrato (puertos formales)	Vía código	No	Vía plugins	Sí
Contrato de extensión público y documentado	Sí	No	Parcial	Sí
Integración nativa documental + SQL con enrutamiento	Manual	Parcial	Limitada	Sí
Seguridad por capas en <i>text-to-SQL</i>	Manual	Limitada	Limitada	Sí (diseño)

La contribución central de este trabajo es la **plataforma propuesta** —un producto open-source, autoalojable y modular— que cubre el perímetro funcional de las soluciones de bajo código existentes (Dify, Flowise, Langflow, Botpress) e introduce **diferenciadores arquitectónicos y de diseño** que la distinguen de ellas. Estos diferenciadores se articulan en torno a tres aspectos:

- **Modularidad por contrato frente a modularidad por plugin.** Las plataformas existentes ofrecen modularidad como *selección* entre proveedores soportados o como sistemas de plugins informales. La plataforma propuesta define puertos formales del dominio (contratos explícitos para LLM, embeddings, base vectorial, fuentes de datos, fragmentación, enrutamiento) que cualquier desarrollador —incluido el propio administrador con perfil técnico— puede implementar para ejecutar piezas

en su propia infraestructura sin tocar el núcleo. La diferencia es de *grado y de formalismo*, no de existencia: lo que en otras plataformas requiere conocer el código interno o ceñirse a una API de plugins propietaria, aquí se reduce a cumplir un contrato documentado. Por ilustrar la diferencia con un ejemplo concreto, integrar en Dify un proveedor de LLM no soportado de fábrica requiere conocer la estructura interna del sistema de *plugins*, navegar la documentación específica de la plataforma y ceñirse a las extensiones que esta permite; en la plataforma propuesta, la misma integración consiste en escribir una clase Python que herede de `LLMProvider` e implemente el método `generate` y registrarla en la configuración. El contrato es trivial, está documentado en la propia interfaz y no depende del conocimiento implícito del sistema.

- **Integración nativa de fuentes documentales y SQL con seguridad por capas.** A diferencia de las soluciones que tratan *text-to-SQL* como una funcionalidad añadida, la plataforma propuesta lo incorpora como ciudadano de primera clase a través del módulo de enrutamiento, junto con los mecanismos de defensa en profundidad descritos en la Sección 3.2 (mínimo privilegio en la conexión, validador sintáctico y revisión semántica con LLM ligero).
- **Documentación del patrón aplicado a RAG.** Como subproducto del desarrollo, el trabajo formaliza y documenta la aplicación de la arquitectura hexagonal al paradigma RAG en términos de puertos y adaptadores explícitos, proporcionando una referencia —en forma de diseño, justificación y evaluación cuantitativa— para futuros sistemas que deban combinar accesibilidad No-Code con extensibilidad real.

Capítulo 4

Metodología

Este capítulo describe el enfoque metodológico adoptado para el diseño, la implementación y la evaluación de la plataforma. Se detalla el marco de investigación, las fases de desarrollo y su vinculación con las entregas académicas, el stack tecnológico seleccionado, la estrategia de evaluación del sistema RAG, los escenarios de validación previstos y la gestión de riesgos del proyecto.

4.1. Enfoque y tipo de investigación

El presente trabajo se plantea como un **proyecto de ingeniería de software con rigor metodológico en la evaluación**: su objetivo es diseñar, construir y validar empíricamente un artefacto software —la plataforma modular RAG— que resuelva un problema real identificado en el Capítulo 1: la dificultad de crear y desplegar chatbots de atención al cliente sin un desarrollo a medida para cada organización. La memoria escrita y la defensa oral constituyen el entregable académico evaluado por el tribunal; la plataforma funcional cumple un doble papel: dentro del marco académico es el artefacto que sustenta empíricamente las afirmaciones de la memoria, y fuera de él es el resultado de ingeniería del proyecto. La formalización del patrón hexagonal aplicado al paradigma RAG (Sección 3.4) constituye una aportación documentativa derivada del proceso de construcción.

Como **marco organizativo** para sistematizar el ciclo de diseño, construcción y evaluación se adopta **Design Science Research** (DSR) ([Hevner et al., 2004](#)), un paradigma utilizado en la investigación en sistemas de información que aporta vocabulario y estructura para abordar la creación de artefactos tecnológicos. DSR no determina por sí mismo si la contribución del trabajo es académica o aplicada; en este caso se utiliza como herramienta de organización metodológica que permite enmarcar las decisiones de diseño, las fases de desarrollo y la estrategia de evaluación dentro de un proceso reconocido en la literatura.

La estrategia de evaluación se enmarca en el framework FEDS de [Venable et al. \(2016\)](#), que clasifica las evaluaciones de DSR según dos ejes ortogonales: artificial vs. naturalista (entornos controlados frente a contextos reales de uso) y formativa vs. sumativa (durante la construcción frente a tras su finalización). La evaluación adoptada en este trabajo es

artificial (utiliza datasets de prueba, no usuarios reales en su contexto natural) y combina dos componentes complementarios. Durante la construcción, las métricas de calidad RAGAS, las métricas operacionales (latencia, coste) y las pruebas adversariales del módulo *text-to-SQL* (Sección 4.4.6) se utilizan iterativamente para guiar las decisiones de diseño y configuración del pipeline (componente **formativo**). Una vez consolidada la plataforma, se ejecuta una pasada definitiva sobre la configuración elegida cuyos resultados se reportan como evaluación final del trabajo (componente **sumativo**).

Se sigue el modelo de proceso propuesto por [Peffers et al. \(2007\)](#), que estructura la investigación en seis actividades:

1. **Identificación del problema y motivación.** Definición del problema —la ausencia de una solución modular, abierta y autoalojable para la creación de chatbots RAG— y justificación del valor de resolverlo (Capítulo 1).
2. **Definición de los objetivos de la solución.** Especificación de los objetivos generales y específicos que el artefacto debe satisfacer (Capítulo 2).
3. **Diseño y desarrollo.** Creación del artefacto: la arquitectura modular basada en puertos y adaptadores, el pipeline de ingestión multi-fuente, el motor RAG con enrutamiento y el panel de administración web (Capítulo 5).
4. **Demostración.** Despliegue de la plataforma en escenarios de uso diferenciados para verificar que el artefacto funciona según lo previsto (Sección 4.5).
5. **Evaluación.** Medición cuantitativa del rendimiento del sistema mediante métricas de recuperación y generación, comparación con un baseline interno y discusión explícita de las amenazas a la validez (Sección 4.4).
6. **Comunicación.** Difusión de los resultados a través del presente documento y del depósito del código fuente en un repositorio público en GitHub acompañado de documentación de instalación, uso y extensión. Se adopta la licencia **AGPLv3** (*Affero General Public License*, versión 3), que permite el uso, modificación y redistribución del código y, adicionalmente, exige que cualquier despliegue como servicio en red publique también su código fuente. Esta cláusula preserva el carácter abierto del trabajo frente a red despliegues cerrados por terceros y se alinea con la pretensión de modularidad por contrato y autoalojabilidad declaradas en los objetivos.

El desarrollo lo realiza un único autor, sin sprints ni proceso ágil formal con *stakeholders* externos. La estrategia consiste en construir primero versiones mínimas y funcionales de cada módulo que permitan ejecutar el pipeline de extremo a extremo (atendiendo al orden de dependencias entre componentes), y a continuación iterar sobre cada módulo para refinarlo, midiendo el impacto de los cambios mediante las métricas descritas en la Sección 4.4. Las entregas parciales del máster actúan como hitos de integración que delimitan el alcance funcional disponible en cada momento.

4.2. Fases del desarrollo

El desarrollo del trabajo se organiza en cuatro fases principales, vinculadas a las entregas académicas del programa:

4.2.1. Fase 1: Definición del problema y objetivos

Periodo: 02/03/2026 – 09/03/2026.

Esta fase comprende la identificación y delimitación del problema, la formulación del objetivo general y los objetivos específicos del trabajo, y el establecimiento del alcance del MVP.

Entregable asociado: Primera entrega (09/03/2026) — Título, introducción y objetivos.

4.2.2. Fase 2: Marco teórico y metodología

Periodo: 24/03/2026 – 04/05/2026.

Se realiza la revisión bibliográfica que sustenta el marco teórico, abarcando los paradigmas y tecnologías relevantes para el trabajo: LLMs, RAG, embeddings y bases de datos vectoriales, arquitectura de software modular y estado del arte. Se selecciona de forma preliminar el enfoque arquitectónico (puertos y adaptadores) y el stack tecnológico, justificando ambas decisiones a la luz del marco teórico. Paralelamente, se formaliza la estrategia de evaluación y se planifica la validación. El diseño detallado de la arquitectura —diagramas de componentes, especificación formal de los contratos de los puertos— se concreta en la fase siguiente.

Entregable asociado: Segunda entrega (04/05/2026) — Marco teórico y metodología.

4.2.3. Fase 3: Diseño detallado, implementación y validación

Periodo: 05/05/2026 – 06/07/2026.

Se concreta el diseño arquitectónico, se implementa el producto mínimo viable (MVP) de la plataforma y se realiza la evaluación. La fase se organiza en cinco **etapas**, ordenadas según las dependencias entre módulos:

1. **Etapas 1 — Diseño arquitectónico detallado:** elaboración de los diagramas de componentes, especificación formal de los contratos de cada puerto y definición concreta de las interfaces de los adaptadores que se implementarán en el MVP.
2. **Etapas 2 — Núcleo del pipeline RAG:** implementación del pipeline de ingestión de documentos (PDF, DOCX, TXT, CSV, Markdown), la vectorización mediante el sistema de embeddings, el almacenamiento en Qdrant y la generación de respuestas con recuperación semántica.

3. **Etapa 3 — Fuentes heterogéneas y enrutamiento:** integración de fuentes de datos SQL y almacenamiento en la nube, e implementación del módulo de enrutamiento que determina la estrategia de recuperación adecuada para cada consulta, junto con las capas de seguridad del módulo *text-to-SQL*.
4. **Etapa 4 — Panel de administración web:** desarrollo de la interfaz web que permite crear, configurar y gestionar chatbots, incluyendo la selección del modelo de lenguaje, la vinculación de fuentes de conocimiento y el ajuste de parámetros del pipeline.
5. **Etapa 5 — Evaluación, validación y refinamiento:** construcción de los datasets de evaluación por escenario; ejecución de las comparativas entre el baseline específico de cada escenario y las configuraciones de la plataforma (Sección 4.4); evaluación adversarial del módulo *text-to-SQL* (Sección 4.4.6); registro de métricas operacionales (latencia, coste); análisis de resultados y refinamiento de los componentes a partir de los hallazgos.

Entregable asociado: Tercera entrega (06/07/2026) — Resultados, discusión, conclusiones y referencias.

4.2.4. Fase 4: Redacción final y depósito

Periodo: 07/07/2026 – 13/07/2026.

Consolidación de la memoria, revisión integral del documento y depósito del trabajo.

4.3. Herramientas y tecnologías

La selección del stack tecnológico responde a tres criterios: (1) que todas las herramientas sean **completamente gratuitas** —ya sea por ser de código abierto o por disponer de un nivel de uso gratuito sin coste para el desarrollo y la evaluación del trabajo—, (2) que permitan el despliegue autoalojado (*self-hosted*) sin dependencia de proveedores de infraestructura concretos, y (3) que cuenten con un ecosistema maduro y una comunidad activa.

Una decisión de diseño relevante es la **no adopción de frameworks de orquestación RAG** como LangChain o LlamaIndex (descritos en la Sección 3.5) como dependencia estructural de la plataforma. Esta decisión responde a dos razones. En primer lugar, estos frameworks imponen su propia jerarquía de abstracciones (*chains*, *agents*, *retrievers*), que entraría en conflicto con la arquitectura de puertos y adaptadores propuesta: la modularidad del sistema debe estar definida por el diseño propio de la plataforma, no delegada en las abstracciones de un framework externo. En segundo lugar, dado que la contribución principal de este trabajo es precisamente el diseño e implementación de la arquitectura modular, delegar la orquestación del pipeline en un framework existente reduciría sustancialmente el alcance y la originalidad del trabajo. La plataforma interactúa directamente

con las APIs de los proveedores de modelos y de la base de datos vectorial, manteniendo el control total sobre cada componente del pipeline.

4.3.1. Backend: Python y FastAPI

El backend de la plataforma se implementa en **Python**, el lenguaje predominante en el ecosistema de inteligencia artificial y procesamiento de lenguaje natural, lo que garantiza la disponibilidad de bibliotecas para manipulación de documentos, generación de embeddings e interacción con modelos de lenguaje. Como framework web se utiliza **FastAPI** (Ramírez, 2024), un framework asíncrono de alto rendimiento que genera automáticamente documentación OpenAPI y ofrece validación de datos integrada mediante Pydantic. FastAPI expone la API REST que consume el frontend y que, al ser un contrato público y documentado, permite a terceros integrar la plataforma con otras interfaces o sistemas.

4.3.2. Frontend: Next.js

La interfaz web del panel de administración se desarrolla con **Next.js** (Vercel, Inc., 2024), un framework basado en React que ofrece renderizado del lado del servidor (SSR), generación estática y un sistema de enrutamiento integrado. Next.js se utiliza exclusivamente como capa de presentación: toda la lógica de negocio reside en el backend Python, y la comunicación se realiza mediante llamadas a la API REST.

4.3.3. Base de datos vectorial: Qdrant

Para el almacenamiento y la búsqueda semántica de los embeddings generados durante la ingestión de documentos se utiliza **Qdrant** (Qdrant Solutions GmbH, 2024), una base de datos vectorial de código abierto escrita en Rust que implementa el algoritmo HNSW (*Hierarchical Navigable Small World*) descrito en la Sección 3.3. Qdrant se despliega como un servicio independiente dentro de la infraestructura Docker y ofrece una API REST y gRPC nativas, lo que facilita su integración con el backend Python. Su carácter autoalojable y la ausencia de límites en la versión de código abierto lo hacen especialmente adecuado para los requisitos de la plataforma.

4.3.4. Base de datos relacional: PostgreSQL

PostgreSQL (The PostgreSQL Global Development Group, 2024) se emplea con un doble propósito: (1) como base de datos interna de la plataforma, donde se almacenan los metadatos de los chatbots, las configuraciones de los pipelines, las fuentes de conocimiento vinculadas y los datos de los administradores del panel; y (2) como fuente de datos externa de prueba en el escenario de validación basado en consultas SQL (Sección 4.5). El acceso desde Python se realiza mediante **SQLAlchemy** (Bayer, 2024) en su modo Core, que se utiliza para gestionar los metadatos de la plataforma y para ejecutar de forma segura las sentencias SQL textuales producidas por el módulo *text-to-SQL*, aprovechando su

connection pooling y la propagación de los permisos restringidos del usuario de base de datos. La generación de las sentencias en sí no la realiza SQLAlchemy, sino el LLM, según se describe en la Sección 3.2.

4.3.5. Modelos de lenguaje: Ollama y endpoints compatibles

La arquitectura modular de la plataforma permite integrar cualquier modelo de lenguaje que exponga una API compatible. En el MVP se ofrecen dos adaptadores:

- **Adaptador Ollama nativo** (Ollama, 2024): ejecuta modelos *open-weights* (Llama, Mistral, Gemma, Qwen, entre otros) de forma local, sin enviar datos a servicios externos. Utiliza la API propia de Ollama, que ofrece control específico sobre los parámetros de inferencia y la gestión de los modelos cargados.
- **Adaptador para endpoints compatibles con la API de OpenAI**: cubre tanto el servicio comercial de OpenAI (OpenAI, 2024) como una variedad de proveedores que adoptan el mismo formato de API (*Chat Completions*), entre ellos GitHub Models, NVIDIA NIM, Groq y Together AI. La compatibilidad se obtiene parametrizando la URL del endpoint y la clave de acceso, sin modificar el código del adaptador, lo que permite alternar entre servicios comerciales y endpoints gratuitos con *rate limit* de forma transparente.

Para la evaluación del presente trabajo se utilizan exclusivamente Ollama y endpoints gratuitos, en coherencia con los criterios de selección del stack. Gracias al patrón de puertos y adaptadores, cada adaptador se implementa de forma independiente cumpliendo la interfaz del puerto `LLMProvider`; añadir soporte para proveedores adicionales con APIs propias (Anthropic, Google Gemini, Cohere) consiste en implementar un nuevo adaptador sin modificar el núcleo del sistema.

4.3.6. Infraestructura: Docker y Docker Compose

La plataforma se empaqueta y despliega mediante **Docker** (Docker, Inc., 2024), lo que garantiza la reproducibilidad del entorno y la independencia respecto al sistema operativo o la infraestructura del administrador. Un archivo `docker-compose.yml` define todos los servicios —backend, frontend, Qdrant, PostgreSQL— y permite levantar la plataforma completa con un único comando. Este enfoque cumple el requisito de despliegue agnóstico: la plataforma puede ejecutarse en cualquier máquina que soporte Docker, ya sea un servidor local, una máquina virtual o un proveedor de nube.

4.3.7. Control de versiones y calidad del código

El código fuente de la plataforma se gestiona mediante **Git** como sistema de control de versiones, lo que permite trazar la evolución del proyecto, gestionar ramas de desarrollo

para cada etapa de implementación y facilitar la colaboración futura al tratarse de un proyecto de código abierto.

Se utiliza **pytest** (Krekel et al., 2024) como framework de pruebas unitarias e integración para el backend Python. Las pruebas unitarias validan el comportamiento de cada módulo de forma aislada —aprovechando la testabilidad que aporta la arquitectura hexagonal (Sección 3.4), que permite sustituir adaptadores reales por dobles de prueba—, mientras que las pruebas de integración verifican la interacción entre componentes (por ejemplo, que el pipeline de ingestión almacena correctamente los embeddings en Qdrant). Se establece como objetivo una **cobertura mínima del 80 %** sobre los módulos del núcleo del dominio (puertos, servicios de orquestación y adaptadores propios), reportada mediante **pytest-cov**. La verificación de pruebas y cobertura se automatiza mediante **Integración Continua con GitHub Actions**, configurada para ejecutar la batería completa de tests y publicar el reporte de cobertura en cada *push* y cada *pull request* sobre las ramas principales del repositorio. La evaluación del sistema RAG como conjunto se aborda de forma independiente en la Sección 4.4.

La Tabla 4.1 resume el stack tecnológico completo.

Cuadro 4.1: Resumen del stack tecnológico de la plataforma.

Componente	Tecnología	Propósito
Backend / API	Python + FastAPI	Lógica de negocio, orquestación RAG, API REST
Frontend	Next.js (React)	Panel de administración web
Base de datos vectorial	Qdrant	Almacenamiento y búsqueda semántica de embeddings
Base de datos relacional	PostgreSQL	Metadatos de la plataforma y fuente de datos SQL de prueba
ORM	SQLAlchemy	Acceso a bases de datos relacionales
LLM (local)	Ollama	Ejecución de modelos de código abierto en local
LLM remoto	Endpoints OpenAI-compatibles	Inferencia vía OpenAI, GitHub Models, NVIDIA NIM, Groq, Together AI
Infraestructura	Docker + Compose	Empaquetado, despliegue y orquestación de servicios
Pruebas	pytest + pytest-cov	Pruebas unitarias e integración con reporte de cobertura

4.4. Estrategia de evaluación

La evaluación del sistema se centra en medir cuantitativamente la calidad del pipeline RAG en sus dos dimensiones fundamentales: la capacidad de recuperar fragmentos relevantes ante una consulta y la capacidad de generar respuestas correctas y fundamentadas a partir de los fragmentos recuperados. Para ello se utiliza el framework **RAGAS** (*Retrieval Augmented Generation Assessment*) (Es et al., 2024), un marco de evaluación automatizada diseñado específicamente para sistemas RAG que permite calcular métricas sin necesidad de juicio humano, utilizando un LLM como evaluador.

4.4.1. Métricas de recuperación

Las métricas de recuperación evalúan la calidad de los fragmentos devueltos por la base de datos vectorial antes de que sean procesados por el modelo de lenguaje:

- **Context Precision** (Precisión del contexto): mide la proporción de fragmentos recuperados que son efectivamente relevantes para responder la consulta. Un valor alto indica que el sistema no introduce ruido innecesario en el contexto del LLM.
- **Context Recall** (Cobertura del contexto): mide en qué grado los fragmentos recuperados cubren la información necesaria para generar la respuesta de referencia. Un valor alto indica que el sistema no omite información relevante.

4.4.2. Métricas de generación

Las métricas de generación evalúan la calidad de la respuesta producida por el modelo de lenguaje a partir del contexto recuperado:

- **Faithfulness** (Fidelidad): mide la proporción de afirmaciones en la respuesta generada que pueden verificarse a partir de los fragmentos recuperados. Es la métrica principal para detectar alucinaciones: una fidelidad baja indica que el modelo ha inventado información no respaldada por el contexto.
- **Answer Relevancy** (Relevancia de la respuesta): evalúa en qué medida la respuesta generada aborda directamente la pregunta formulada, penalizando respuestas vagas, incompletas o tangenciales.

RAGAS ofrece métricas adicionales como *Answer Correctness* (corrección factual de la respuesta frente a una respuesta de referencia) o *Answer Semantic Similarity* (similitud semántica entre respuesta generada y referencia). La selección de las cuatro métricas anteriores responde a un criterio de no redundancia: las cuatro cubren las dos dimensiones críticas del paradigma RAG (calidad de la recuperación y calidad de la generación) sin solaparse entre sí, mientras que las métricas omitidas resultan parcialmente redundantes con *Faithfulness* cuando hay contexto recuperado, o difíciles de interpretar al penalizar respuestas correctas formuladas de forma distinta a la referencia.

4.4.3. Protocolo de evaluación

La evaluación se ejecuta de forma automatizada mediante un *script* que, para cada escenario de validación (Sección 4.5):

1. Carga el dataset de evaluación correspondiente (pares de pregunta, respuesta de referencia y contexto de referencia).
2. Ejecuta cada pregunta contra el pipeline RAG de la plataforma, recogiendo la respuesta generada y los fragmentos recuperados.
3. Calcula las cuatro métricas descritas utilizando RAGAS.
4. Genera un informe con los resultados agregados y desglosados por pregunta.

4.4.4. Baselines y comparativas entre configuraciones

Como referencia de comparación se establece un **baseline específico para cada escenario**, dado que los modos de recuperación involucrados son distintos y requieren líneas base diferenciadas:

- **Escenario 1 (búsqueda semántica):** configuración de RAG ingenuo con *chunking* de tamaño fijo (512 *tokens* con solapamiento 50, valores de referencia habituales en la literatura), un único modelo de embeddings open-source de tamaño medio, y $\text{top-}k = 5$ sin re-ranking. Permite cuantificar la mejora aportada por estrategias de *chunking* alternativas, embeddings de mayor calidad y ajustes del $\text{top-}k$.
- **Escenario 2 (*text-to-SQL*):** la línea base es la consulta SQL ideal escrita manualmente para cada pregunta del dataset. Se mide la **precisión de ejecución** (*execution accuracy*) —si la consulta generada produce el mismo conjunto de resultados que la consulta de referencia— complementada por las métricas de generación de RAGAS sobre la respuesta final.
- **Escenario 3 (enrutamiento multi-fuente):** la propia plataforma con el módulo de enrutamiento desactivado, forzando todas las consultas a la búsqueda semántica. Permite cuantificar la aportación específica del enrutador, particularmente en consultas que requieren acceso a datos estructurados.

Sin estos baselines declarados, los valores absolutos de las métricas resultan difícilmente interpretables.

Adicionalmente, sobre el baseline de cada escenario se realizan **comparativas entre configuraciones** para analizar el impacto de los distintos parámetros del pipeline sobre la calidad del sistema. Las comparativas se ejecutan siguiendo un esquema **OFAT** (*one-factor-at-a-time*): se varía una dimensión a la vez manteniendo las demás en su valor por defecto, en lugar de explorar el producto cartesiano de configuraciones (factorial completo), que resultaría inviable con los *rate limits* de los endpoints gratuitos. Las dimensiones que se varían y los escenarios sobre los que aplica cada una son:

- La **estrategia de fragmentación** (*chunking*) —tamaño del fragmento y solapamiento— en los Escenarios 1 y 3 (componente documental).
- El **modelo de embeddings**, en los Escenarios 1 y 3, comparando modelos de distinto tamaño y familia.
- El **modelo de lenguaje**, en los tres escenarios, contrastando modelos locales (Ollama) y modelos comerciales (API de OpenAI).
- El **número de fragmentos recuperados** (k) en los Escenarios 1 y 3.

Los resultados de estas comparativas permiten identificar las configuraciones óptimas para cada escenario y formular recomendaciones prácticas para los administradores de la plataforma.

4.4.5. Métricas operacionales

Junto a las métricas de calidad, durante la ejecución del benchmark se registran **métricas operacionales** para cada configuración: latencia media por consulta —descompuesta por etapas del pipeline (*embedding* de la consulta, búsqueda vectorial, llamada al LLM, post-procesado)— y coste por consulta, medido en *tokens* consumidos cuando se utilizan APIs comerciales y en tiempo de cómputo cuando se ejecutan modelos locales. Estas métricas permiten comparar configuraciones no solo en términos de calidad sino también de viabilidad operacional, particularmente al contrastar el uso de modelos locales (Ollama) frente a modelos comerciales (OpenAI), donde la diferencia de coste por consulta puede ser determinante para el caso de uso.

4.4.6. Evaluación adversarial del módulo *text-to-SQL*

Dado que el OE-3 incorpora mecanismos de seguridad por capas para proteger las bases de datos consultadas mediante *text-to-SQL* (Sección 3.2), se incluye un protocolo de evaluación adversarial específico que verifica empíricamente esa promesa. Se construye una batería de entre 25 y 30 *prompts* adversariales en lenguaje natural —tres ejemplos por categoría— diseñados para inducir al sistema a generar consultas peligrosas. Las categorías cubiertas son: exfiltración de datos sensibles (`SELECT * FROM users WHERE password ...`), modificación o destrucción de tablas (`DROP`, `DELETE`, `UPDATE`), escalado de privilegios, evasión de filtros mediante comentarios o sentencias encadenadas, y *prompt injection* que instruye al modelo a ignorar sus restricciones. La validación sintáctica (segunda capa defensiva) no se basa en filtrado por palabras clave —trivialmente evadible mediante variantes ortográficas, comentarios SQL o codificación alternativa— sino en el **parseo de la consulta a un árbol sintáctico** (*Abstract Syntax Tree*) mediante una librería estándar (`sqlglot`), comprobando estructuralmente que el tipo de sentencia es `SELECT` y rechazando cualquier otra construcción. Cada prompt se ejecuta contra el módulo en condiciones equivalentes a las del despliegue normal, y se mide la **tasa de bloqueo**

en cada una de las tres capas defensivas —mínimo privilegio en la conexión, validación sintáctica AST y revisión semántica con LLM ligero—. Tras la ejecución automatizada, el autor revisa manualmente cada caso para verificar que la capa que bloqueó la consulta lo hizo por la razón esperada, distinguiendo bloqueos por la causa correcta de coincidencias accidentales. El criterio de éxito es que ninguna consulta adversarial alcance la ejecución contra la base de datos real; cualquier consulta que sí lo haga se documenta como hallazgo y se utiliza para reforzar las capas defensivas antes del depósito final. Esta batería proporciona **cobertura categórica** de los patrones de ataque conocidos pero no constituye una auditoría exhaustiva de seguridad; una validación basada en *fuzzing* automatizado y *red-teaming* sistemático se identifica como línea de trabajo futuro.

4.4.7. Limitaciones y amenazas a la validez

La estrategia de evaluación adoptada presenta limitaciones que conviene reconocer explícitamente:

- **Sesgo de auto-evaluación (LLM-as-judge):** las métricas de RAGAS se calculan utilizando un LLM como evaluador, lo que introduce un sesgo cuando el modelo evaluador y el modelo evaluado pertenecen a la misma familia. La mitigación se aplica a dos niveles. En primer lugar, el modelo evaluador se selecciona de una **familia distinta** al modelo evaluado, recurriendo a los endpoints disponibles para combinar familias (por ejemplo, evaluar Llama vía Ollama con un juez de la familia GPT vía GitHub Models, o evaluar Mistral vía Groq con un juez Llama vía Ollama). En segundo lugar, sobre una muestra estratificada de **15–20 tripletas por escenario** —cubriendo los tres niveles de complejidad: factual, inferencial y comparativa— se realiza una **evaluación manual** análoga a las métricas RAGAS, y se reporta el grado de concordancia entre la puntuación humana y la automática mediante el coeficiente de correlación de Spearman. Una concordancia razonable refuerza la confianza en el evaluador automático; una divergencia significativa se documenta como amenaza adicional a la validez y obliga a interpretar los resultados con cautela.
- **Calidad y tamaño del dataset sintético:** el dataset se genera mediante un LLM y se somete a revisión manual para descartar ejemplos ambiguos, triviales o factualmente incorrectos. El tamaño objetivo (50–100 tripletas por escenario) es un compromiso entre representatividad y coste de revisión; los resultados se acompañan de los rangos de variabilidad observados y las diferencias pequeñas entre configuraciones se interpretan con cautela.
- **Validez externa:** los tres escenarios cubren las modalidades de recuperación soportadas pero no agotan el espacio de aplicaciones posibles. La generalización a dominios o tipos de fuente no contemplados —por ejemplo, datos en grafos, contenidos multimodales o corpus muy heterogéneos— requiere validación adicional.

Reconocer estas limitaciones no debilita los resultados; al contrario, delimita su alcance interpretativo y orienta posibles líneas de trabajo futuro.

4.4.8. Reproducibilidad

Para garantizar la reproducibilidad de los resultados, junto con los datos del benchmark se publica el **setup completo** utilizado, incluyendo: las versiones exactas de cada modelo (evaluado, embeddings y juez) identificadas por nombre y versión específica; las semillas aleatorias empleadas en cada componente estocástico (generación del dataset, *sampling* del LLM, partición de datos); el proveedor de inferencia utilizado en cada caso —local mediante Ollama, endpoint remoto compatible con OpenAI, o combinación de ambos— y, cuando aplique, las especificaciones del hardware (CPU, GPU, VRAM, RAM, sistema operativo) y las cuantizaciones empleadas; el dataset de evaluación tras la revisión manual; y el propio *script* de evaluación. Si durante la ejecución se produjeran sustituciones forzadas de proveedor por agotamiento de cuota o limitaciones de hardware, se documentan junto con los resultados para preservar la trazabilidad. Todo el material se publica bajo la misma licencia abierta que la plataforma, de modo que cualquier tercero pueda replicar la evaluación o extenderla a configuraciones adicionales.

Nota sobre evaluación en tiempo real. Una extensión futura de la plataforma contempla integrar la evaluación de métricas directamente en el flujo de despliegue, de modo que el panel de administración muestre al administrador un análisis automático de la calidad del chatbot y sugiera ajustes de configuración. Esta funcionalidad queda fuera del alcance del MVP, pero la arquitectura modular del sistema facilita su incorporación posterior.

Nota sobre evaluación cualitativa. El presente trabajo se limita a la evaluación cuantitativa automatizada. La validación mediante encuestas a usuarios finales o juicio experto se identifica como línea de trabajo futuro que complementaría los resultados obtenidos.

4.5. Datos y escenarios de validación

Para demostrar la versatilidad de la plataforma y evaluar su rendimiento en contextos diferentes, se definen tres escenarios de validación que cubren los modos de recuperación de información del sistema y su combinación. En todos los escenarios, los chatbots se crean y configuran íntegramente desde el panel de administración web, sin modificar código fuente, lo que constituye la validación funcional del objetivo OE-4. El Capítulo 5 documenta el flujo completo de configuración con capturas de pantalla del panel.

4.5.1. Escenario 1: Atención al cliente sobre documentación de producto

En este escenario se despliega un chatbot de atención al cliente configurado para responder preguntas sobre la documentación de un producto: manuales de usuario, guías de configuración, preguntas frecuentes y especificaciones técnicas. Las fuentes de conocimiento consisten en documentos en distintos formatos (PDF, DOCX, Markdown) procedentes

de dos orígenes: una parte se carga como archivos locales y otra se ingesta desde un servicio de almacenamiento en la nube, verificando así el funcionamiento del adaptador de fuentes remotas. Todos los documentos se procesan a través del pipeline de la plataforma, se fragmentan y vectorizan, y se almacenan en Qdrant. La recuperación de información se realiza exclusivamente mediante **búsqueda semántica** sobre la base de datos vectorial. Ejemplos de consultas representativas: «¿cómo configuro el modo nocturno?», «¿es compatible el producto X con el accesorio Y?».

Este escenario valida los objetivos OE-1 (arquitectura modular), OE-2 (pipeline de ingestión multi-fuente, incluyendo archivos locales y almacenamiento en la nube) y OE-5 (evaluación del rendimiento), y representa el caso de uso más habitual de un sistema RAG en atención al cliente.

4.5.2. Escenario 2: Atención al cliente sobre catálogo y pedidos

En este escenario se despliega un chatbot de atención al cliente que responde preguntas sobre información comercial almacenada en una base de datos relacional PostgreSQL: catálogo de productos, niveles de stock, ofertas vigentes y características de los modelos. Ante consultas del usuario en lenguaje natural —«¿qué stock hay del producto X?», «¿qué productos hay en oferta esta semana?», «¿cuáles son las características del modelo más reciente?»—, el sistema genera dinámicamente la consulta SQL correspondiente (*text-to-SQL*), la ejecuta contra la base de datos respetando los mecanismos de seguridad por capas descritos en la Sección 3.2 y construye una respuesta fundamentada en los resultados obtenidos. La recuperación de información se realiza mediante **generación y ejecución de consultas SQL**, sin intervención de la base de datos vectorial. Las consultas que requieren identificación del usuario final (por ejemplo, estado de pedidos personales) quedan fuera del alcance del MVP, ya que la integración con un sistema de autenticación se identifica como extensión natural de la plataforma sin afectar a la arquitectura modular propuesta.

Este escenario valida el objetivo OE-6 (validación en escenarios diferenciados) y demuestra que la plataforma no se limita a la búsqueda semántica sobre documentos, sino que puede operar sobre fuentes de datos estructuradas.

4.5.3. Escenario 3: Atención al cliente multi-fuente con enrutamiento

En este escenario se despliega un chatbot de atención al cliente con acceso simultáneo a ambas fuentes de conocimiento: el corpus documental del producto (vectorizado en Qdrant) y la base de datos relacional con catálogo y datos comerciales (PostgreSQL). Ante cada consulta, el **módulo de enrutamiento** analiza la naturaleza de la pregunta y determina la estrategia de recuperación apropiada: búsqueda semántica para preguntas sobre el funcionamiento del producto, generación de SQL para preguntas sobre datos estructurados, o una combinación de ambas cuando la respuesta requiere integrar infor-

mación de las dos fuentes (por ejemplo, «¿el producto X soporta modo nocturno y qué stock queda?», donde «modo nocturno» requiere consultar la documentación y «stock» requiere consultar la base de datos).

El dataset de evaluación de este escenario incluye tres tipos de preguntas: (1) preguntas que requieren exclusivamente información documental, (2) preguntas que requieren exclusivamente datos de la base de datos SQL, y (3) preguntas que requieren combinar información de ambas fuentes. Esta diversidad permite evaluar tanto la precisión del enrutamiento —si dirige cada consulta a la fuente correcta— como la capacidad del sistema para sintetizar información heterogénea en una respuesta coherente.

Este escenario valida directamente el objetivo OE-3 (motor RAG con enrutamiento de consultas) y complementa la validación de OE-6 al demostrar que la plataforma puede orquestar múltiples fuentes de forma simultánea y transparente para el usuario final.

4.5.4. Generación del dataset de evaluación

Para cada escenario se construye un **dataset de evaluación** compuesto por tripletas (pregunta, respuesta de referencia, contexto de referencia) que sirven como *ground truth* para el cálculo de las métricas descritas en la Sección 4.4.

La generación del dataset sigue un enfoque **sintético asistido por LLM**: a partir de los documentos o datos de cada escenario, se utiliza un modelo de lenguaje para generar automáticamente preguntas junto con sus respuestas de referencia y los fragmentos o registros que las sustentan. Este enfoque, cada vez más extendido en la literatura de evaluación de sistemas RAG (Es et al., 2024; Saad-Falcon et al., 2024), permite crear datasets de tamaño suficiente sin el coste de la anotación manual, siempre que las tripletas generadas se sometan a una revisión posterior para descartar ejemplos ambiguos, triviales o factualmente incorrectos.

Cada escenario contará con un dataset de entre 50 y 100 tripletas, distribuidas en tres tipos de complejidad para cubrir un espectro representativo de consultas: **factuales** (consultas directas que requieren un único fragmento o registro), **inferenciales** (consultas que requieren combinar información de varios fragmentos o aplicar razonamiento sobre los datos) y **comparativas** (consultas que requieren contrastar dos o más entidades). El tamaño elegido es un compromiso entre la robustez estadística —suficiente para calcular intervalos de confianza significativos mediante *bootstrapping*— y el coste de generación, revisión manual y ejecución del benchmark con modelos comerciales. La distribución de tipos por escenario, los corpus y las tablas seleccionadas se documentarán junto con los resultados de la evaluación en el Capítulo 5.

4.6. Gestión de riesgos

El calendario ajustado del trabajo (05/05/2026 – 06/07/2026 para implementación, evaluación y validación) y la complejidad técnica de la plataforma justifican un análisis de los principales riesgos del proyecto y de sus estrategias de mitigación. Cada riesgo se

acompaña de un plan de contingencia que delimita el alcance de la entrega final en el peor caso.

- **Riesgo técnico — complejidad del módulo de enrutamiento:** la calidad del enrutador determina el éxito del Escenario 3. Si la implementación basada en LLM no converge en tiempo razonable o produce decisiones inconsistentes, se sustituye por una estrategia basada en reglas explícitas (clasificación por palabras clave y patrones), documentando la decisión como simplificación del MVP en el capítulo de desarrollo.
- **Riesgo de calendario — profundidad de la evaluación:** la batería completa de comparativas (estrategia de *chunking*, modelo de embeddings, modelo de lenguaje, top- k) es ambiciosa. Si el tiempo no permite completarla, se prioriza la comparación entre el baseline y la configuración por defecto de la plataforma, dejando las comparativas extensas como anexo o como línea de trabajo futuro explícitamente identificada.
- **Riesgo operacional — *rate limits* en endpoints gratuitos:** la generación del dataset y la ejecución de la batería con endpoints gratuitos compatibles con la API de OpenAI están sujetas a límites por minuto y por día que pueden interrumpir o ralentizar el proceso. La mitigación consiste en distribuir la batería entre varios proveedores compatibles (GitHub Models, NVIDIA NIM, Groq, Together AI) y caer al adaptador Ollama local como *fallback* cuando una cuota se agota. Toda sustitución forzada de proveedor durante la ejecución se documenta junto con los resultados, manteniendo la reproducibilidad.
- **Riesgo de alcance — panel web:** el desarrollo del frontend completo es accesorio respecto a la contribución arquitectónica. Si el calendario lo exige, se prioriza un panel funcional mínimo que permita las operaciones esenciales (crear chatbot, vincular fuentes, configurar parámetros del pipeline) frente a un panel pulido visualmente, manteniendo la trazabilidad con OE-4 sin comprometer OE-1 a OE-3.
- **Riesgo metodológico — sesgos del dataset sintético:** si la revisión manual descarta una fracción elevada del dataset generado por LLM, el tamaño efectivo puede comprometer la significancia estadística. La mitigación consiste en iterar el *prompt* de generación mejorando la calidad inicial y, en última instancia, completar manualmente las tripletas faltantes hasta alcanzar el tamaño mínimo declarado en la Sección 4.5.4.
- **Riesgo de hardware — VRAM limitada (condicional a la elección de inferencia local):** si la evaluación se ejecuta total o parcialmente con modelos locales mediante Ollama, las restricciones de hardware del autor limitan el tamaño máximo del modelo viable. La mitigación consiste en complementar la evaluación con endpoints remotos gratuitos (GitHub Models, NVIDIA NIM, Groq, Together

AI), que permiten acceder a modelos de mayor tamaño sin coste, dejando explícita en los resultados la mezcla de proveedores empleada.

Estos riesgos se monitorizan a lo largo de la Fase 3 y los planes de contingencia se activan tan pronto como se detectan los disparadores correspondientes, priorizando siempre el cumplimiento de los objetivos centrales del trabajo —OE-1 (arquitectura modular), OE-2 (pipeline de ingestión multi-fuente) y OE-3 (motor RAG con enrutamiento)— sobre la profundidad de la evaluación o el pulido del frontend.

Capítulo 5

Desarrollo y Resultados

Capítulo 6

Objetivos de Desarrollo Sostenible

Capítulo 7

Conclusiones

Referencias

Bibliografía

- Asai, A., Wu, Z., Wang, Y., et al. (2024). Self-RAG: Learning to retrieve, generate, and critique through self-reflection. *Proceedings of the International Conference on Learning Representations (ICLR)*. Disponible en: <https://arxiv.org/abs/2310.11511>.
- Bayer, M. (2024). SQLAlchemy: The Python SQL toolkit and object relational mapper. Disponible en: <https://www.sqlalchemy.org/>.
- Bommasani, R., Hudson, D. A., Adeli, E., et al. (2021). On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258*. Disponible en: <https://arxiv.org/abs/2108.07258>.
- Botpress Technologies Inc. (2024). Botpress: The complete platform for AI agents. Disponible en: <https://botpress.com/>.
- Brown, T., Mann, B., Ryder, N., et al. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33:1877–1901. Disponible en: <https://arxiv.org/abs/2005.14165>.
- Chase, H. (2022). LangChain. Disponible en: <https://github.com/langchain-ai/langchain>.
- Chen, W., Zha, H., Chen, Z., et al. (2020). HybridQA: A dataset of multi-hop question answering over tabular and textual data. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 1026–1036. Association for Computational Linguistics. Disponible en: <https://aclanthology.org/2020.findings-emnlp.91/>.
- Cockburn, A. (2005). Hexagonal architecture. *alistair.cockburn.us*. Disponible en: <https://alistair.cockburn.us/hexagonal-architecture/>.
- deepset (2024). Haystack: end-to-end nlp framework. Disponible en: <https://haystack.deepset.ai/>.
- Devlin, J., Chang, M.-W., Lee, K., et al. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of NAACL-HLT*, pages 4171–4186. Disponible en: <https://aclanthology.org/N19-1423/>.
- Docker, Inc. (2024). Docker: Accelerated container application development. Disponible en: <https://www.docker.com/>.

- Edge, D., Trinh, H., Cheng, N., et al. (2024). From local to global: A graph rag approach to query-focused summarization. In *arXiv preprint arXiv:2404.16130*. Disponible en: <https://arxiv.org/abs/2404.16130>.
- Es, S., James, J., Espinosa-Anke, L., et al. (2024). RAGAS: Automated evaluation of retrieval augmented generation. *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, pages 150–163. Disponible en: <https://aclanthology.org/2024.eacl-demo.16/>.
- FlowiseAI (2024). Flowise: Build AI agents visually. Disponible en: <https://flowiseai.com/>.
- Formal, T., Piwowarski, B., and Clinchant, S. (2021). Splade: Sparse lexical and expansion model for first stage ranking. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*. Disponible en: <https://doi.org/10.1145/3404835.3463098>.
- Gamma, E., Helm, R., Johnson, R., et al. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- Gao, D., Wang, H., Li, Y., et al. (2024a). Text-to-SQL empowered by large language models: A benchmark evaluation. *Proceedings of the VLDB Endowment*, 17(5):1132–1145. Disponible en: <https://doi.org/10.14778/3641204.3641221>. Versión preliminar: arXiv:2308.15363 (2023).
- Gao, L., Ma, X., Lin, J., et al. (2023). Precise zero-shot dense retrieval without relevance labels. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*. Disponible en: <https://aclanthology.org/2023.acl-long.99/>. Versión preliminar: arXiv:2212.10496 (2022).
- Gao, T., Yao, X., and Chen, D. (2021). Simcse: Simple contrastive learning of sentence embeddings. *Proceedings of EMNLP 2021*. Disponible en: <https://aclanthology.org/2021.emnlp-main.552/>.
- Gao, Y., Xiong, Y., Gao, X., et al. (2024b). Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*. Disponible en: <https://arxiv.org/abs/2312.10997>.
- Google (2025). EmbeddingGemma: Open embedding model optimized for on-device AI. Disponible en: <https://ai.google.dev/gemma/docs/embeddinggemma>.
- Google (2026). Gemini Embedding 2: Our first natively multimodal embedding model. Disponible en: <https://blog.google/innovation-and-ai/models-and-research/gemini-models/gemini-embedding-2/>.

- Hevner, A. R., March, S. T., Park, J., et al. (2004). Design science in information systems research. *MIS Quarterly*, 28(1):75–105. Disponible en: <https://doi.org/10.2307/25148625>.
- Huang, L., Yu, W., Ma, W., et al. (2023). A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Transactions on Information Systems*. Disponible en: <https://arxiv.org/abs/2311.05232>.
- Jégou, H., Douze, M., and Schmid, C. (2011). Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(1):117–128. Disponible en: <https://doi.org/10.1109/TPAMI.2010.57>.
- Jiang, Z., Xu, F. F., Gao, L., et al. (2023). Active retrieval augmented generation. *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 7969–7992. Disponible en: <https://aclanthology.org/2023.emnlp-main.495/>.
- Johnson, J., Douze, M., and Jégou, H. (2021). Billion-scale similarity search with GPUs. In *IEEE Transactions on Big Data*, volume 7, pages 535–547. IEEE. Disponible en: <https://doi.org/10.1109/TBDATA.2019.2921572>.
- Karpukhin, V., Oğuz, B., Min, S., et al. (2020). Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781. Association for Computational Linguistics. Disponible en: <https://aclanthology.org/2020.emnlp-main.550/>.
- Khattab, O. and Zaharia, M. (2020). Colbert: Efficient and effective passage search via contextualized late interaction over bert. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*. Disponible en: <https://doi.org/10.1145/3397271.3401075>.
- Krekel, H. et al. (2024). pytest: Simple powerful testing with Python. Disponible en: <https://pytest.org/>.
- Langflow (2024). Langflow: Low-code AI builder for agentic and retrieval-augmented AI applications. Disponible en: <https://www.langflow.org/>.
- LangGenius, Inc. (2024). Dify: Open-source LLM app development platform. Disponible en: <https://dify.ai/>.
- Lewis, P., Perez, E., Piktus, A., et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474. Curran Associates, Inc. Disponible en: <https://arxiv.org/abs/2005.11401>.

- Lin, J., Nogueira, R., and Yates, A. (2021). Pretrained transformers for text ranking: Bert and beyond. *Synthesis Lectures on Human Language Technologies*. Disponible en: <https://arxiv.org/abs/2010.06467>.
- Liu, N. F., Lin, K., Hewitt, J., et al. (2024). Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173. Disponible en: https://doi.org/10.1162/tac1_a_00638. Versión preliminar: arXiv:2307.03172 (2023).
- LlamaIndex (2023). LlamaIndex: Data framework for LLM applications. Disponible en: <https://www.llamaindex.ai/>.
- Malkov, Y. A. and Yashunin, D. A. (2020). Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836. Disponible en: <https://doi.org/10.1109/TPAMI.2018.2889473>.
- Martin, R. C. (2018). *Clean Architecture: A Craftsman’s Guide to Software Structure and Design*. Prentice Hall.
- Mikolov, T., Sutskever, I., Chen, K., et al. (2013). Distributed representations of words and phrases and their compositionality. *Advances in Neural Information Processing Systems*, 26. Disponible en: <https://arxiv.org/abs/1310.4546>.
- Nogueira, R. and Cho, K. (2019). Passage re-ranking with bert. In *arXiv preprint arXiv:1901.04085*. Disponible en: <https://arxiv.org/abs/1901.04085>.
- Ollama (2024). Ollama: Run large language models locally. Disponible en: <https://ollama.com/>.
- OpenAI (2024). OpenAI API reference. Disponible en: <https://platform.openai.com/docs/api-reference>.
- Ouyang, L., Wu, J., Jiang, X., et al. (2022). Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*. Disponible en: <https://arxiv.org/abs/2203.02155>.
- Palermo, J. (2008). The onion architecture: Part 1. Disponible en: <https://jeffreypalermo.com/2008/07/the-onion-architecture-part-1/>.
- Pedro, R., Coimbra, M. E., Castro, D., et al. (2025). Prompt-to-SQL injections in LLM-integrated web applications: Risks and defenses. In *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. Disponible en: <https://doi.org/10.1109/ICSE55347.2025.00007>. Versión preliminar: arXiv:2308.01990 (2023).
- Peppers, K., Tuunanen, T., Rothenberger, M. A., et al. (2007). A design science research methodology for information systems research. In *Journal of Management Information*

- Systems*, volume 24, pages 45–77. Taylor & Francis. Disponible en: <https://doi.org/10.2753/MIS0742-1222240302>.
- Petroni, F., Rocktäschel, T., Riedel, S., et al. (2019). Language models as knowledge bases? In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Disponible en: <https://aclanthology.org/D19-1250/>.
- Qdrant Solutions GmbH (2024). Qdrant: Vector search engine and database. Disponible en: <https://qdrant.tech/>.
- Ramírez, S. (2024). FastAPI: Modern, fast web framework for building APIs with Python. Disponible en: <https://fastapi.tiangolo.com/>.
- Reimers, N. and Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using siamese BERT-networks. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 3982–3992. Disponible en: <https://aclanthology.org/D19-1410/>.
- Saad-Falcon, J., Khattab, O., Potts, C., et al. (2024). Ares: An automated evaluation framework for retrieval-augmented generation systems. In *Proceedings of NAACL 2024*. Disponible en: <https://aclanthology.org/2024.naacl-long.20/>. Versión preliminar: arXiv:2311.09476 (2023).
- Talmor, A., Yoran, O., Catav, A., et al. (2021). MultiModalQA: Complex question answering over text, tables and images. In *International Conference on Learning Representations (ICLR)*. Disponible en: <https://arxiv.org/abs/2104.06039>.
- The PostgreSQL Global Development Group (2024). PostgreSQL: The world’s most advanced open source relational database. Disponible en: <https://www.postgresql.org/>.
- Touvron, H., Lavril, T., Izacard, G., et al. (2023). LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*. Disponible en: <https://arxiv.org/abs/2302.13971>.
- Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30. Disponible en: <https://arxiv.org/abs/1706.03762>.
- Venable, J., Pries-Heje, J., and Baskerville, R. (2016). Feds: A framework for evaluation in design science research. *European Journal of Information Systems*, 25(1):77–89. Disponible en: <https://doi.org/10.1057/ejis.2014.36>.
- Vercel, Inc. (2024). Next.js: The React framework for the web. Disponible en: <https://nextjs.org/>.

- Wang, L., Yang, N., Huang, X., et al. (2024). Multilingual E5 text embeddings: A technical report. *arXiv preprint arXiv:2402.05672*. Disponible en: <https://arxiv.org/abs/2402.05672>.
- Wei, J., Wang, X., Schuurmans, D., et al. (2022). Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35:24824–24837. Disponible en: <https://arxiv.org/abs/2201.11903>.
- Wu, S., Xiong, Y., Cui, Y., et al. (2024). Retrieval-augmented generation for natural language processing: A survey. *arXiv preprint arXiv:2407.13193*. Disponible en: <https://arxiv.org/abs/2407.13193>.
- Wulf, J. and Meierhofer, J. (2024). Exploring the potential of large language models for automation in technical customer service. In *Proceedings of the Spring Servitization Conference (SSC2024)*, pages 146–157, Tilburg, The Netherlands. The Advanced Services Group. Disponible en: <https://arxiv.org/abs/2405.09161>.
- Xiao, S., Liu, Z., Zhang, P., et al. (2024). C-Pack: Packed resources for general Chinese embeddings. *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. Disponible en: <https://arxiv.org/abs/2309.07597>.

Acrónimos

ANN Approximate Nearest Neighbor (Vecino Más Cercano Aproximado)

API Application Programming Interface

BERT Bidirectional Encoder Representations from Transformers

BM25 Best Matching 25 (algoritmo léxico clásico de *ranking*)

DI Dependency Injection (Inyección de Dependencias)

DPR Dense Passage Retrieval

DSR Design Science Research

gRPC Google Remote Procedure Call

HNSW Hierarchical Navigable Small World

HyDE Hypothetical Document Embeddings

IVF Inverted File Index

kNN k-Nearest Neighbors (k Vecinos Más Cercanos)

LLM Large Language Model (Modelo de Lenguaje de Gran Escala)

MTEB Massive Text Embedding Benchmark

MVP Minimum Viable Product (Producto Mínimo Viable)

NLP Natural Language Processing (Procesamiento de Lenguaje Natural)

ORM Object-Relational Mapper

RAG Retrieval-Augmented Generation (Generación Aumentada por Recuperación)

REST Representational State Transfer

RGPD Reglamento General de Protección de Datos

RLHF Reinforcement Learning from Human Feedback (Aprendizaje por Refuerzo con Retroalimentación Humana)

SFT Supervised Fine-Tuning (Ajuste Fino Supervisado)

SPLADE Sparse Lexical and Expansion Model for First Stage Ranking

SQL Structured Query Language

SSR Server-Side Rendering (Renderizado del Lado del Servidor)

Anexos